

Guide de défense orale

Comprendre, expliquer et défendre le projet de A à Z

Romain · Yannis · Lisa

github.com/RomainJazzar/travelling-merchant

Guide de défense orale — « Le marchand ambulant »

Projet TSP · 20 villes françaises · Christofides vs Algorithme génétique Romain · Yannis · Lisa
Dépôt : github.com/RomainJazzar/travelling-merchant

Comment utiliser ce document

Ce guide est conçu pour qu'une personne qui ne connaît **rien** au projet puisse, en le lisant de bout en bout, le comprendre, l'expliquer et le défendre devant un professeur.

Il distingue systématiquement cinq registres — ne les confondez jamais à l'oral :

Registre	Exemple	Formulation à l'oral
Ce que le sujet demande	« Vous considérez la distance de Haversine »	« Le sujet impose... »
Ce que fait notre code	Paris est fixé comme ancre du chromosome	« Dans notre implémentation... »
Ce que dit la théorie	Christofides $\leq 1,5 \times \text{OPT}$ en TSP métrique	« Le théorème garantit... »
Ce que montrent nos mesures	3 157,13 km sur nos 3 seeds	« Nous avons observé... »
Ce que nous proposons	ajouter du 2-opt	« Une amélioration possible serait... »

Règle d'or : ne jamais transformer une mesure en théorème, ni un choix d'implémentation en nécessité mathématique.

Toutes les valeurs chiffrées de ce guide proviennent de l'exécution réelle de `python main.py` et sont stockées dans `results/summary.json`. Elles ont été reproduites à l'identique lors de la rédaction de ce guide.

Les chiffres du projet (vérifiés à l'exécution)

Grandeur	Valeur	Source
Villes	20	<code>data/villes_france_lat_long.csv</code>
Arêtes du graphe complet	190	$C(20,2)$
Rayon terrestre utilisé	6 371,0088 km	<code>src/core.py</code>
Christofides — distance	3 445,60 km	<code>summary.json</code>
Poids du MST (Prim)	2 665,05 km	<code>summary.json</code>
Poids du couplage minimal	1 142,03 km	<code>summary.json</code>
Sommets de degré impair	12	<code>summary.json</code>
Génétique — meilleure distance	3 157,13 km	<code>summary.json</code>
Écart GA — Christofides	−288,48 km	<code>summary.json</code>
Amélioration relative	−8,37 %	<code>summary.json</code>
Configuration GA retenue	<code>equilibree</code>	choisie par <code>main.py</code>

Grandeur	Valeur	Source
Population / générations	160 / 520	<code>summary.json</code>
Tournoi / croisement / mutation / élite	5 / 95 % / 22 % / 6	<code>summary.json</code>
Seeds testés	11, 22, 33	<code>main.py</code>
Génération où le meilleur est atteint (seed 11)	67 sur 520	mesuré
Temps Christofides observé	$\approx 1,1 - 2,2$ ms	mesuré, dépend de la machine
Temps GA observé (une exécution)	$\approx 0,7 - 3,0$ s	mesuré, dépend de la machine

⚠ **Les temps d'exécution sont des mesures expérimentales, pas des constantes.** Sur la machine de développement initiale, Christofides tournait en $\approx 1,6$ ms et le GA en $\approx 0,68$ s ; lors d'une exécution ultérieure sur une machine plus chargée, les mêmes calculs ont pris $\approx 1,5$ ms et $\approx 2,95$ s. **Les distances, elles, sont rigoureusement identiques** — c'est ce qui compte. Si le professeur relève un écart de temps entre le README et votre démo, la bonne réponse est : « le temps dépend de la machine et de sa charge ; ce qui est reproductible, ce sont les distances, parce que le seed est fixé. »

PARTIE 1 — Comprendre le sujet

1.1 L'histoire, puis le problème

Théobald est un marchand ambulant de la France médiévale. Il doit visiter un ensemble de marchés, une fois chacun, et rentrer chez lui. Chaque kilomètre inutile coûte de l'argent, du temps, et l'expose aux bandits et aux intempéries.

Traduit en langage algorithmique, cela devient :

Étant donné 20 villes et la distance entre chaque paire, trouver l'ordre de visite qui minimise la distance totale d'un circuit fermé passant une fois par chaque ville.

C'est le **problème du voyageur de commerce** (Travelling Salesman Problem, TSP).

1.2 Vocabulaire de base

Permutation. Un arrangement ordonné de tous les éléments d'un ensemble. `[Paris, Lyon, Nice]` et `[Paris, Nice, Lyon]` sont deux permutations différentes des mêmes trois villes. Une tournée TSP **est** une permutation des villes.

Factorielle. $n! = n \times (n-1) \times \dots \times 2 \times 1$. C'est le nombre de permutations de n éléments. $5! = 120$, $10! = 3\,628\,800$, $19! \approx 1,216 \times 10^{17}$.

Espace de recherche. L'ensemble de toutes les solutions candidates. Ici : toutes les tournées possibles.

Optimum global. La meilleure solution de tout l'espace de recherche.

Optimum local. Une solution meilleure que toutes celles de son voisinage immédiat, mais pas forcément la meilleure de l'espace entier. Image : vous êtes au sommet d'une colline ; tout autour descend, mais il existe une montagne plus haute ailleurs.

Solution approchée (heuristique). Une solution obtenue sans garantie d'être optimale, mais calculable en temps raisonnable.

Algorithme d'approximation. Une heuristique qui, en plus, **garantit** un facteur maximal d'écart à l'optimum. Christofides en est un ; un algorithme génétique **n'en est pas un**.

1.3 Pourquoi $(n-1)!$ et pas $n!$?

Le sujet écrit explicitement $(n-1)!$. Raison : dans un **cycle**, le point de départ n'a pas d'importance. La tournée

Paris → Lyon → Nice → Paris

et la tournée

Lyon → Nice → Paris → Lyon

sont le même cycle, simplement écrit à partir d'un autre point. Il y a n façons d'écrire le même cycle (une par ville de départ), donc on divise $n!$ par n , ce qui donne $(n-1)!$.

Avec $n = 20$: **$19! = 121\,645\,100\,408\,832\,000$** , soit environ **$1,216 \times 10^{17}$** .

Le raffinement que le professeur peut demander

Il existe une seconde symétrie : **le sens de parcours**. Comme notre graphe est non orienté, la tournée **Paris → Lyon → Nice → Paris** a exactement la même longueur que **Paris → Nice → Lyon → Paris**. Elles sont donc identiques du point de vue du coût.

Le nombre de cycles **réellement distincts** est donc :

$$(n-1)! / 2 = 19! / 2 = 60\,822\,550\,204\,416\,000 \approx 6,08 \times 10^{16}$$

Comment le dire : « Le sujet donne $(n-1)!$, soit $19!$. Si on tient compte en plus du sens de parcours, qui ne change pas la longueur dans un graphe non orienté, on descend à $19!/2$. Diviser par deux ne change rien au message : on reste à plus de 6×10^{16} tournées, donc la force brute est hors de portée. »

L'ordre de grandeur, concrètement

Supposons une machine capable d'évaluer **un milliard de tournées par seconde** (ce qui est déjà très optimiste en Python) :

$$1,216 \times 10^{17} / 10^9 = 1,216 \times 10^8 \text{ secondes} \approx 3,9 \text{ ans}$$

Pour **une seule instance de 20 villes**. Et en ajoutant une seule ville, on multiplie ce temps par 20.

1.4 NP-difficile : le dire correctement

Le sujet indique que le TSP est **NP-difficile**. Voici comment le formuler sans se tromper :

« NP-difficile signifie qu'on ne connaît aucun algorithme qui résout le problème exactement en temps polynomial, et qu'un tel algorithme, s'il existait, en fournirait aussi un pour tous les problèmes de la classe NP. »

Nuances importantes à ne PAS rater :

- NP-difficile n'est **pas** la même chose que « impossible ». Des solveurs exacts

(Concorde, branch-and-cut) résolvent régulièrement des instances de milliers de villes. Ce qui manque, c'est une garantie de temps polynomial **dans tous les cas**.

- La version *décision* du TSP (« existe-t-il une tournée de longueur $\leq L$? ») est

NP-complète. La version *optimisation* (« trouvez la plus courte ») est **NP-difficile**. Si on vous demande la différence, c'est celle-ci.

- $P = NP$ est un problème ouvert. Ne dites jamais « on a prouvé qu'il n'existe pas d'algorithme rapide » — on ne l'a pas prouvé.

1.5 Ce que le sujet demande exactement

Reprenons le sujet point par point, pour vérifier que rien ne manque :

Exigence du sujet	Où c'est traité
Récupérer les positions de 20 villes françaises	data/villes_france_lat_long.csv
Représenter le réseau sous forme de graphe	src/core.py::build_complete_graph
Sommets = villes, arêtes = routes, poids = distances	idem
Utiliser la distance de Haversine	src/core.py::haversine_km
Implémenter Christofides	src/christofides.py
Donner la distance totale	3 445,60 km
Afficher l'itinéraire sur la carte	results/route_christofides.html (+ PNG)
Expliquer les étapes et justifier la pertinence	Partie 6 de ce guide
Implémenter un algorithme génétique	src/genetic.py
Définir les paramètres (population, sélection, reproduction, mutation, générations)	Partie 10
Tester différentes configurations	benchmark_configs , 3 configs × 3 seeds
Donner la distance totale	3 157,13 km
Afficher l'itinéraire sur la carte	results/route_genetique.html (+ PNG)
Comparer : distance, temps, facilité d'implémentation, robustesse	Partie 20
Avantages / inconvénients dans le contexte de Théobald	Partie 20 + 26
Recommandation finale	Partie 21 + conclusion
Présentation incluant le Trello	slide 18 du PowerPoint
Dépôt GitHub public nommé travelling-merchant	✓
README avec contexte, algorithmes, conclusion	✓

PARTIE 2 — Théorie des graphes : tout le vocabulaire

Pour chaque notion : **définition** · **exemple** · **rôle dans le projet** · **question probable**.

2.1 Graphe

Définition. Un couple $G = (V, E)$ où V est un ensemble de **sommets** et E un ensemble d'**arêtes** reliant des paires de sommets.

Exemple. $V = \{\text{Paris, Lyon, Nice}\}$, $E = \{(\text{Paris,Lyon}), (\text{Lyon,Nice}), (\text{Paris,Nice})\}$.

Rôle ici. C'est la structure de tout le projet. `build_complete_graph` en construit un avec NetworkX.

Question probable. « *Qu'est-ce qu'un graphe ?* » → « Un ensemble de sommets et un ensemble de liens entre ces sommets. Ici, les sommets sont les 20 villes. »

2.2 Sommet (vertex / node)

Définition. Un élément de V .

Rôle ici. Une ville. Dans le code, un sommet est un **entier** (0 à 19) ; le nom, la latitude et la longitude sont stockés comme attributs :

```
graph.add_node(i, name=city.name, latitude=city.latitude, longitude=city.longitude)
```

Question probable. « *Pourquoi des entiers plutôt que des noms ?* » → « Parce que les indices servent directement à adresser la matrice des distances : `matrix[i][j]`. C'est plus rapide et plus simple que des clés textuelles. »

2.3 Arête (edge)

Définition. Un lien entre deux sommets.

Rôle ici. Une liaison possible entre deux villes.

2.4 Poids

Définition. Une valeur numérique associée à une arête.

Rôle ici. La distance de Haversine en kilomètres.

Question probable. « *Que représente le poids ?* » → « La distance à vol d'oiseau entre les deux villes, en kilomètres. »

2.5 Graphe orienté vs non orienté

Orienté. Les arêtes ont un sens : $(A \rightarrow B) \neq (B \rightarrow A)$. On parle d'**arcs**.

Non orienté. L'arête $\{A, B\}$ se parcourt dans les deux sens.

Rôle ici. Notre graphe est **non orienté**, parce que Haversine est symétrique : $d(A, B) = d(B, A)$. `nx.Graph()` est la classe non orientée de NetworkX (`nx.DiGraph` serait la version orientée).

Question probable. « *Et si les routes étaient à sens unique, ou si l'aller montait et le retour descendait ?* » → « Il faudrait un graphe orienté, et on ne serait plus dans un TSP symétrique. Christofides ne s'appliquerait plus tel quel : la version asymétrique du TSP a d'autres algorithmes d'approximation, bien moins bons. »

2.6 Graphe pondéré

Définition. Graphe dont chaque arête porte un poids.

Rôle ici. Indispensable : sans poids, « le chemin le plus court » n'a pas de sens.

2.7 Graphe complet

Définition. Graphe où **toute** paire de sommets distincts est reliée par une arête. Noté K_n .

Nombre d'arêtes. $C(n, 2) = n(n-1)/2$. Pour $n = 20$: **190**.

Rôle ici. On peut calculer Haversine entre n'importe quelle paire de villes, donc toutes les liaisons existent.

Question probable. « Pourquoi 190 et pas 380 ? » → « Parce que le graphe est non orienté : l'arête Paris-Lyon et l'arête Lyon-Paris sont la même. $20 \times 19 = 380$ compte les couples ordonnés ; on divise par 2 pour obtenir les paires. »

Question piège. « Et si deux villes n'étaient pas reliées par une route ? » → « Le graphe ne serait plus complet, et il pourrait ne plus exister de cycle hamiltonien du tout. C'est justement parce qu'on modélise des distances géodésiques, et non le réseau routier, que le graphe est complet. »

2.8 Graphe connexe

Définition. Il existe un chemin entre toute paire de sommets.

Rôle ici. Un graphe complet est trivialement connexe. C'est ce qui garantit que Prim peut atteindre tous les sommets — `prim_mst` lève d'ailleurs une exception si ce n'était pas le cas :

```
if len(visited) != graph.number_of_nodes():
    raise ValueError("Le graphe n'est pas connexe.")
```

2.9 Sous-graphe

Définition. Un graphe $H = (V', E')$ avec $V' \subseteq V$ et $E' \subseteq E$.

Rôle ici. Le MST est un sous-graphe du graphe complet. Le graphe des sommets impairs aussi (c'est le sous-graphe **induit** par ces 12 sommets).

2.10 Chemin, cycle, circuit

- **Chemin** : suite de sommets reliés par des arêtes, sans répétition de sommet.
- **Cycle** : un chemin qui revient à son point de départ.
- **Circuit** : terme souvent employé pour un cycle dans lequel on s'intéresse aux

arêtes (circuit eulérien). En français, « cycle » insiste sur les sommets, « circuit » sur le parcours des arêtes. Les deux mots se recoupent ; ce qui compte, c'est de préciser **eulérien** ou **hamiltonien**.

Rôle ici. On cherche un **cycle hamiltonien de poids minimal**.

2.11 Degré d'un sommet

Définition. Le nombre d'arêtes incidentes à ce sommet.

Exemple. Dans notre MST, Paris a un certain degré ; 12 villes ont un degré impair.

Rôle ici. C'est la notion pivot de Christofides. Un sommet de degré impair empêche l'existence d'un circuit eulérien.

Question probable. « Pourquoi le degré vous intéresse-t-il ? » → « Parce que pour traverser un sommet sans s'y arrêter, il faut y entrer par une arête et en ressortir par une autre : les arêtes se consomment par paires. Un sommet de degré impair finit donc toujours par bloquer le parcours. »

2.12 Lemme des poignées de main

Énoncé. Dans tout graphe, la somme des degrés vaut deux fois le nombre d'arêtes :

$$\sum \deg(v) = 2 |E|$$

Pourquoi. Chaque arête a deux extrémités et contribue donc 1 au degré de chacune : elle est comptée exactement deux fois dans la somme.

Conséquence capitale. $2|E|$ est pair. Or une somme paire ne peut pas contenir un nombre **impair** de termes impairs (impair + impair = pair ; il faut donc qu'ils s'apparient). Donc :

Le nombre de sommets de degré impair est toujours pair.

Rôle ici. C'est ce qui rend le couplage parfait possible. Nous avons 12 sommets impairs — un nombre pair, comme le théorème l'exige.

2.13 Arbre

Définition. Un graphe **connexe** et **sans cycle**.

Propriété. Un arbre à n sommets a exactement $n - 1$ arêtes. Ajouter une arête crée un cycle ; en retirer une déconnecte le graphe.

Arbre enraciné. Un arbre où un sommet est désigné comme racine, ce qui oriente les relations parent/enfant. **Nous n'en utilisons pas** : notre MST n'a pas de racine, il est simplement non orienté. (Prim part d'un sommet, mais ce n'est pas une racine au sens structurel.)

Arbre binaire. Un arbre enraciné où chaque nœud a au plus deux enfants. **Rien à voir avec ce projet** — sauf que le *tas binaire* (`heapq`) utilisé par Prim en est un. C'est un piège classique : ne confondez pas *arbre couvrant* et *arbre binaire*.

2.14 Arbre couvrant (spanning tree)

Définition. Un sous-graphe qui est un arbre **et** qui touche **tous** les sommets.

Exemple. Avec 20 villes, c'est un réseau de 19 liaisons qui connecte tout le monde sans boucle.

2.15 Arbre couvrant minimal (MST)

Définition. L'arbre couvrant dont la somme des poids est minimale.

Valeur ici. 2 665,05 km, 19 arêtes.

Question probable. « *Le MST est-il unique ?* » → « Pas nécessairement en général : s'il y a des poids égaux, plusieurs MST peuvent exister. Mais ils ont tous le même poids total. Avec des distances géodésiques réelles, les égalités exactes sont quasiment impossibles, donc le nôtre est de fait unique. Nous avons d'ailleurs vérifié que Prim donne le même poids quel que soit le sommet de départ, et que ce poids coïncide avec celui de `networkx.minimum_spanning_tree`. »

2.16 Circuit eulérien

Définition. Un circuit fermé qui emprunte **chaque arête exactement une fois**.

Théorème d'Euler. Un graphe connexe possède un circuit eulérien **si et seulement si** tous ses sommets sont de degré pair.

Rôle ici. C'est l'objet intermédiaire de Christofides : MST + couplage donne un multigraphe connexe à degrés tous pairs, donc eulérien.

2.17 Cycle hamiltonien

Définition. Un cycle qui passe par **chaque sommet exactement une fois** et revient au départ.

Rôle ici. C'est exactement ce que cherche le TSP.

Différence à retenir absolument.

	Euler	Hamilton
Objet compté	les arêtes	les sommets
Condition d'existence	degrés tous pairs (facile à tester)	aucun critère simple (problème NP-complet)
Algorithme	Hierholzer, en $O(\sum d_i)$	$E \cdot V$) pas d'algorithme efficace connu

Question piège. « Si trouver un cycle hamiltonien est NP-complet, comment votre programme en trouve un en quelques millisecondes ? » → « Parce que dans un graphe **complet**, l'existence d'un cycle hamiltonien est triviale : n'importe quelle permutation en est un. Ce qui est difficile, ce n'est pas d'en trouver un, c'est d'en trouver le **plus court**. »

2.18 Multigraphe

Définition. Un graphe qui autorise plusieurs arêtes entre la même paire de sommets (arêtes parallèles), et éventuellement des boucles.

Rôle ici. L'union MST + couplage est un multigraphe : une arête du couplage peut déjà être présente dans le MST, auquel cas elle apparaît deux fois.

Comment le code le gère. `_hierholzer_multigraph` ne manipule pas des paires de sommets mais des **identifiants d'arête** :

```
for edge_id, (u, v, _) in enumerate(all_edges):
    adjacency[u].append((v, edge_id))
    adjacency[v].append((u, edge_id))
```

Chaque arête a son propre `edge_id`, donc deux arêtes parallèles restent distinctes. C'est précisément pour cela qu'on n'a pas utilisé `nx.Graph` ici : `nx.Graph` fusionne les arêtes parallèles, ce qui casserait la parité des degrés.

Question probable. « Pourquoi une liste d'adjacence maison et pas NetworkX ? » → « Parce qu'il nous faut un multigraphe, et que nous voulions maîtriser exactement la gestion des arêtes parallèles dans Hierholzer. »

2.19 Couplage (matching)

Couplage. Un ensemble d'arêtes **sans sommet commun**.

Couplage parfait. Un couplage qui couvre **tous** les sommets — chaque sommet appartient à exactement une arête. Il n'existe que si le nombre de sommets est pair.

Couplage parfait de poids minimal. Parmi tous les couplages parfaits, celui dont la somme des poids est la plus faible.

Valeur ici. 6 arêtes, **1 142,03 km**, sur les 12 sommets impairs :

Paire appariée	Distance
Lyon – Grenoble	94,33 km
Saint-Étienne – Clermont-Ferrand	107,88 km
Lille – Reims	167,62 km
Nice – Nîmes	233,41 km
Strasbourg – Dijon	245,17 km
Bordeaux – Angers	293,62 km
Total	1 142,03 km

Question probable. « Pourquoi "parfait" ? » → « Parce que **tous** les sommets impairs doivent être corrigés. Si on en laissait un de côté, il resterait de degré impair et il n'y aurait pas de circuit eulérien. »

Question probable. « Pourquoi "de poids minimal" ? » → « Pour deux raisons. La première est pratique : les arêtes qu'on ajoute s'ajoutent au coût final, donc on veut les moins chères. La seconde est théorique : c'est la minimalité qui permet de prouver que le couplage coûte au plus OPT/2, et c'est cette borne qui donne le facteur 1,5. »

2.20 Borne inférieure et borne supérieure

Borne inférieure de l'optimum. Une valeur dont on est sûr que l'optimum **ne descend pas en dessous**. Ici : le poids du MST, **2 665,05 km**.

Borne supérieure de l'optimum. Une valeur dont on est sûr que l'optimum **ne monte pas au-dessus**. N'importe quelle tournée valide en est une : la meilleure que nous ayons trouvée, **3 157,13 km**.

Ce que nous savons donc :

$$2\,665,05 \leq \text{OPT} \leq 3\,157,13 \quad (\text{fenêtre de } 492,07 \text{ km})$$

Question probable. « Connaissez-vous l'optimum ? » → « Non. Nous savons qu'il est dans cette fenêtre, et c'est tout ce que nous pouvons affirmer honnêtement. »

PARTIE 3 — Haversine et la notion de métrique

3.1 Latitude et longitude

- **Latitude (ϕ)** : position nord-sud, de -90° (pôle Sud) à $+90^\circ$ (pôle Nord).

L'équateur est à 0° . Paris est à $48,8566^\circ$ N.

- **Longitude (λ)** : position est-ouest, de -180° à $+180^\circ$, comptée depuis le méridien

de Greenwich. Paris est à $2,3522^\circ$ E.

Point crucial. Un degré de latitude vaut toujours ≈ 111 km. Un degré de **longitude**, lui, vaut ≈ 111 km à l'équateur mais seulement $\approx 111 \times \cos(48,86^\circ) \approx 73$ km à la latitude de Paris. Les méridiens se resserrent vers les pôles.

Conséquence. Appliquer Pythagore directement sur (Δlat , Δlon) en degrés revient à additionner des unités qui ne valent pas la même chose. C'est faux.

3.2 Radians

Les fonctions trigonométriques de Python (`math.sin`, `math.cos`) attendent des **radians**, pas des degrés. D'où la première ligne de notre fonction :

```
lat1, lon1 = math.radians(a.latitude), math.radians(a.longitude)
```

Conversion. `radians = degrés ×  $\pi$  / 180`.

Question piège. « Que se passerait-il si vous oubliiez `math.radians` ? » → « Les distances seraient complètement fausses — `sin(48.8566)` interprété en radians n'a aucun rapport avec `sin(48,8566°)`. »

C'est l'erreur la plus classique sur Haversine. »

3.3 Le rayon terrestre

Nous utilisons `EARTH_RADIUS_KM = 6371.0088`.

C'est le **rayon moyen volumétrique** de la Terre défini par l'IUGG. La Terre n'est pas une sphère parfaite : son rayon équatorial vaut $\approx 6\,378$ km et son rayon polaire $\approx 6\,357$ km. Prendre 6 371,0088 est le compromis standard.

Question probable. « *Pourquoi cette valeur si précise ?* » → « C'est le rayon moyen officiel. La précision affichée n'améliore pas beaucoup le résultat — c'est la sphéricité supposée qui est la vraie approximation — mais c'est la valeur de référence, donc autant l'utiliser. »

3.4 La formule

$$h = \sin^2(\Delta\varphi/2) + \cos(\varphi_1) \cdot \cos(\varphi_2) \cdot \sin^2(\Delta\lambda/2)$$
$$d = 2 \cdot R \cdot \arcsin(\sqrt{h})$$

Traduite en Python :

```
h = (
    math.sin(dlat / 2) ** 2
    + math.cos(lat1) * math.cos(lat2) * math.sin(dlon / 2) ** 2
)
return 2 * EARTH_RADIUS_KM * math.asin(math.sqrt(h))
```

Intuition. h est le « haversine » de l'angle central entre les deux points ($\text{haversine}(\theta) = \sin^2(\theta/2)$). En prenant $2 \cdot \arcsin(\sqrt{h})$ on récupère cet angle en radians, et en le multipliant par le rayon on obtient la longueur de l'arc.

Pourquoi cette forme et pas la loi des cosinus sphériques ? La formule $\cos(d/R) = \sin \varphi_1 \sin \varphi_2 + \cos \varphi_1 \cos \varphi_2 \cos \Delta\lambda$ est mathématiquement équivalente, mais elle perd énormément de précision numérique pour les **petites** distances, parce que $\arccos$ est mal conditionné près de 1. Haversine reste stable. Comme plusieurs de nos villes sont proches (Lyon–Saint-Étienne : 57 km), c'est un vrai avantage.

3.5 Pourquoi Haversine et pas autre chose ?

Alternative	Pourquoi nous ne l'avons pas prise
Distance euclidienne sur (lat, lon)	Faux : mélange des unités qui ne valent pas la même distance selon la latitude.
Euclidienne après projection (Lambert 93...)	Jouable pour la France, mais ajoute une dépendance et une étape de projection pour un gain nul à cette échelle.
Formule de Vincenty / géodésique ellipsoïdale	Plus précise (tient compte de l'aplatissement), mais plus lourde et le gain est de l'ordre de 0,3 %, sans effet sur le classement des tournées.
API Google Maps / OSRM	Donne des distances routières réelles, mais : dépendance réseau, quotas, non-déterminisme, et surtout ces distances ne garantissent pas l'inégalité triangulaire — Christofides perdrait sa garantie.
Loi des cosinus sphériques	Équivalente mais numériquement instable sur les courtes distances.

3.6 Qu'est-ce qu'une métrique ?

Une fonction $d(x, y)$ est une **distance** (ou métrique) si elle vérifie quatre propriétés :

1. **Non-négativité** : $d(x, y) \geq 0$.
2. **Identité des indiscernables** : $d(x, y) = 0$ si et seulement si $x = y$.
3. **Symétrie** : $d(x, y) = d(y, x)$.
4. **Inégalité triangulaire** : $d(x, z) \leq d(x, y) + d(y, z)$.

Haversine vérifie les quatre : c'est la distance géodésique sur une sphère, qui est l'archétype d'une métrique.

3.7 L'inégalité triangulaire, et pourquoi tout en dépend

Énoncé. Le détour par un point intermédiaire ne peut jamais raccourcir le trajet :

$$d(A, C) \leq d(A, B) + d(B, C)$$

Pourquoi c'est vrai ici. L'arc de grand cercle est, par définition, le plus court chemin entre deux points d'une sphère. Passer par un troisième point est donc au moins aussi long.

Pourquoi Christofides en a absolument besoin. L'étape finale de Christofides consiste à **sauter** les villes déjà visitées dans le circuit eulérien. Sauter B, c'est remplacer $A \rightarrow B \rightarrow C$ par $A \rightarrow C$. Sans inégalité triangulaire, ce remplacement pourrait **allonger** la tournée, et toute la preuve du facteur 1,5 s'effondrerait.

Le TSP métrique. Quand les distances forment une métrique, on parle de **TSP métrique**. C'est exactement notre cas, et c'est ce qui rend Christofides applicable.

Question piège. « Les distances routières respectent-elles l'inégalité triangulaire ? » → « En distance pure, presque toujours. Mais en **temps de trajet**, non : passer par l'autoroute peut être plus rapide qu'une route directe. Et même en distance, des cas particuliers (sens interdits, tunnels fermés) peuvent la violer. C'est une des raisons pour lesquelles nous sommes restés sur Haversine. »

Vérification que nous avons faite. Nous avons testé les 6 840 triplets (i, j, k) de notre matrice : **zéro violation** de l'inégalité triangulaire. Notre instance est donc bien un TSP métrique, ce qui n'est pas une supposition mais un fait vérifié.

3.8 Ce que Haversine n'est pas

Haversine donne la distance **à vol d'oiseau**. Théobald ne vole pas.

- La distance routière réelle Paris-Marseille est ≈ 775 km ; Haversine donne ≈ 661 km.
- Le ratio routier/géodésique en France est typiquement de **1,15 à 1,30**.

Conséquence honnête à assumer : nos 3 157 km sont une **borne basse irréaliste** de ce que Théobald parcourrait vraiment. Mais comme le biais s'applique à **toutes** les tournées, la **comparaison** entre Christofides et le génétique reste valable — c'est elle qui fait l'objet du sujet.

PARTIE 4 — L'arbre couvrant minimal

4.1 Spanning tree vs minimum spanning tree

- **Spanning tree** : n'importe quel arbre qui touche tous les sommets. Il y en a énormément (formule de Cayley : $n^{(n-2)}$ pour un graphe complet, soit $20^{18} \approx 2,6 \times 10^{23}$ pour nous).
- **Minimum spanning tree** : celui dont la somme des poids est la plus faible.

4.2 Pourquoi n sommets $\rightarrow n-1$ arêtes

Raisonnement constructif. Partez d'un seul sommet (0 arête). Pour rattacher chaque sommet supplémentaire sans créer de cycle, il faut exactement une nouvelle arête. Après avoir rattaché les $n-1$ autres sommets, vous avez $n-1$ arêtes.

Vérification par l'absurde. Avec n arêtes ou plus, il existerait forcément un cycle. Avec $n-2$ ou moins, le graphe serait déconnecté.

C'est exactement ce que fait la boucle de `prim_mst` : elle ajoute une arête par sommet nouvellement visité.

4.3 Pourquoi le MST n'est PAS une tournée TSP

C'est une confusion très fréquente. Trois raisons :

1. **Un arbre n'a pas de cycle**, donc il ne revient jamais au point de départ.
2. **Des sommets ont un degré 1** (les feuilles) : on y arrive et on ne peut plus en repartir sans rebrousser chemin.
3. **Des sommets ont un degré ≥ 3** : dans une tournée, chaque ville a exactement deux voisins (celui d'avant et celui d'après). Le MST a des embranchements.

❑ Ne dites **jamais** : « le MST est une tournée de 2 665 km ». ❑ Dites : « le MST est une structure de connexion, pas un circuit. Son poids sert de borne inférieure. »

4.4 Pourquoi $MST \leq OPT$ (preuve intuitive)

C'est l'argument le plus important de toute la partie Christofides. Apprenez-le.

1. Soit T^* la tournée optimale du TSP. C'est un cycle qui touche les n sommets et comporte n arêtes.
2. Retirez **une seule arête quelconque** de T^* . Vous obtenez un **chemin** qui passe encore par tous les sommets.
3. Un chemin couvrant est un cas particulier d'**arbre couvrant** (il est connexe, il touche tout, il n'a pas de cycle).
4. Donc son poids est $\geq$ au poids du **minimum** spanning tree, par définition du minimum.
5. Or son poids est $\leq$ au poids de T^* (on a retiré une arête de poids ≥ 0).

D'où :

$$\text{poids}(MST) \leq \text{poids}(T^* \text{ privée d'une arête}) \leq \text{poids}(T^*) = OPT$$

Chez nous : $2\,665,05 \leq OPT$. C'est une information **prouvée**, pas une estimation.

Question probable. « Cette borne est-elle serrée ? » $\rightarrow$ « Pas très. Un MST est typiquement 10 à 25 % en dessous de l'optimum du TSP. Il existe de meilleures bornes inférieures — notamment la borne de Held-Karp, obtenue par relaxation lagrangienne, qui atteint souvent 99 % de l'optimum — mais elles sont bien plus coûteuses à calculer et sortaient du cadre du sujet. »

PARTIE 5 — L'algorithme de Prim

5.1 Le principe en une phrase

On fait grossir un arbre à partir d'un sommet, en ajoutant à chaque étape l'arête la moins chère qui relie l'arbre à un sommet pas encore atteint.

C'est un algorithme **glouton** : à chaque étape on prend le meilleur choix local, et on démontre que cela conduit à l'optimum global (ce qui est rare, et c'est ce qui rend le MST si agréable).

5.2 Étape par étape, sur notre code

```
def prim_mst(graph: nx.Graph, start: int = 0) -> list[tuple[int, int, float]]:
    if start not in graph:
        raise ValueError("Sommet de départ absent du graphe.")

    import heapq

    visited = {start}                                # (1)
    heap: list[tuple[float, int, int]] = []
    for neighbor, attrs in graph[start].items():     # (2)
        heapq.heappush(heap, (float(attrs["weight"]), start, neighbor))

    edges: list[tuple[int, int, float]] = []
    while heap and len(visited) < graph.number_of_nodes(): # (3)
        weight, u, v = heapq.heappop(heap)           # (4)
        if v in visited:                             # (5)
            continue
        visited.add(v)
        edges.append((u, v, weight))                  # (6)
        for nxt, attrs in graph[v].items():          # (7)
            if nxt not in visited:
                heapq.heappush(heap, (float(attrs["weight"]), v, nxt))

    if len(visited) != graph.number_of_nodes():      # (8)
        raise ValueError("Le graphe n'est pas connexe.")
    return edges
```

1. **visited** contient les sommets déjà dans l'arbre. On démarre avec **start** (Paris, index 0, par défaut).
2. On met dans le tas toutes les arêtes sortant de **start**.
3. On boucle tant qu'il reste des candidats **et** que l'arbre n'est pas complet.
4. **heappop** retire l'arête de **poids minimal** du tas. C'est le cœur glouton.
5. **Test de fraîcheur** : si **v** est déjà dans l'arbre, cette arête créerait un cycle, on la jette. C'est la technique de la « suppression paresseuse » (*lazy deletion*) : plutôt que de retirer les arêtes périmées du tas, on les ignore à la sortie.
6. Sinon on l'accepte : **v** rejoint l'arbre.
7. On empile les nouvelles arêtes accessibles depuis **v**.
8. Garde-fou : si on n'a pas atteint tous les sommets, le graphe n'était pas connexe.

5.3 Le tas binaire (**heapq**)

Ce que c'est. Une structure qui maintient un ensemble d'éléments et permet de récupérer **le plus petit** en $O(\log n)$, et d'en insérer un en $O(\log n)$.

Pourquoi ici. Sans tas, il faudrait parcourir toutes les arêtes candidates à chaque étape pour trouver la moins chère — $O(E)$ par étape. Le tas ramène cela à $O(\log E)$.

Détail d'implémentation. **heapq** compare les tuples élément par élément. En poussant (**poids**, **u**, **v**), le tri se fait d'abord sur le poids — exactement ce qu'on veut. (Les indices **u**, **v** ne servent de départage

qu'en cas d'égalité parfaite de poids, ce qui n'arrive jamais avec des distances géodésiques.)

Question probable. « *Un tas binaire, c'est un arbre binaire ?* » → « Oui, c'est un arbre binaire presque complet stocké dans un tableau, avec la propriété que chaque parent est plus petit que ses enfants. Mais attention : ça n'a rien à voir avec l'arbre couvrant qu'on est en train de construire. C'est juste l'outil qui sert à le construire. »

5.4 Complexité

Version	Complexité
Prim naïf (tableau)	$O(V^2)$
Prim avec tas binaire (notre cas)	$O(E \log V)$
Prim avec tas de Fibonacci	$O(E + V \log V)$

Sur un graphe **complet**, $E = O(V^2)$, donc notre version est en $O(V^2 \log V)$. Pour $V = 20 : 190$ arêtes, environ $190 \times \log_2(20) \approx 820$ opérations de tas. Instantané.

Remarque intéressante : sur un graphe complet, Prim naïf en $O(V^2)$ serait en réalité **asymptotiquement meilleur** que notre version en $O(V^2 \log V)$. À 20 villes la différence est totalement invisible, et la version avec tas est celle qu'on écrit par défaut parce qu'elle reste efficace sur les graphes creux. C'est une réponse honnête et nuancée si on vous pose la question.

5.5 Prim vs Kruskal vs Dijkstra

	Prim	Kruskal	Dijkstra
But	MST	MST	plus courts chemins depuis une source
Ce qu'il minimise	le coût total de connexion	le coût total de connexion	la distance depuis un sommet fixé
Stratégie	fait grossir un seul arbre	trie toutes les arêtes, fusionne des forêts	relaxe les distances de proche en proche
Structure clé	tas de priorité	union-find	tas de priorité
Complexité	$O(E \log V)$	$O(E \log E)$	$O(E \log V)$
Bon quand	graphe dense	graphe creux	on veut un itinéraire A→B

Pourquoi Prim et pas Kruskal ?

Les deux donnent un MST de **poids identique**. Trois arguments en faveur de Prim ici :

1. Notre graphe est **complet**, donc dense. Prim est mieux adapté aux graphes denses ; Kruskal doit trier les 190 arêtes d'emblée.
2. Prim ne demande qu'un tas ; Kruskal demande en plus une structure **union-find** pour détecter les cycles. C'est une brique de plus à implémenter et à expliquer.
3. Le sujet cite explicitement la ressource « *Comprendre l'Algorithme de Prim et son application dans les graphes* » dans sa base de connaissances.

Réponse honnête si on insiste : « *Kruskal aurait parfaitement convenu et aurait donné exactement le même poids. Le choix de Prim est justifié par la densité du graphe et par la base de connaissances du sujet, pas par une supériorité mathématique.* »

Pourquoi pas Dijkstra ?

Parce que Dijkstra ne résout pas le même problème. Dijkstra répond à « quel est le plus court chemin de Paris à Marseille ? ». Nous, on veut « quel réseau relie toutes les villes au coût total minimal ? ». Ce sont deux objectifs différents.

Contre-exemple mental : l'arbre des plus courts chemins depuis Paris relie chaque ville à Paris de la façon la plus directe, quitte à multiplier les longues liaisons radiales. Le MST, lui, peut relier Nice à Marseille plutôt qu'à Paris, ce qui coûte beaucoup moins cher au total. Les deux arbres sont généralement différents.

5.6 Le sommet de départ change-t-il le résultat ?

Non, pas le poids. Nous l'avons vérifié empiriquement : en lançant `prim_mst` avec chacun des 20 sommets comme point de départ, le poids total est **toujours 2 665,052352 km**.

Justification théorique. Quand les poids sont tous distincts, le MST est unique, donc a fortiori indépendant du point de départ. Prim est correct quel que soit le sommet initial.

Nuance à garder. Si des poids étaient égaux, Prim pourrait renvoyer des MST différents (mais toujours de même poids) selon le départ ou l'ordre d'insertion.

PARTIE 6 — Christofides, étape par étape

6.1 La chaîne complète

```
Graphe complet pondéré (20 sommets, 190 arêtes)
  ↓ Prim
Arbre couvrant minimal – 2 665,05 km, 19 arêtes
  ↓ calcul des degrés
12 sommets de degré impair
  ↓ couplage parfait de poids minimal
6 arêtes – 1 142,03 km
  ↓ union MST u couplage
Multigraphe connexe, tous degrés pairs (25 arêtes)
  ↓ Hierholzer
Circuit eulérien (chaque arête une fois, sommets répétés)
  ↓ shortcutting
Cycle hamiltonien – 3 445,60 km
```

6.2 Pour chaque étape : pourquoi, et que se passerait-il sans elle ?

Étape 1 — MST (Prim)

Pourquoi. Il faut un squelette qui touche toutes les villes au coût le plus bas possible. C'est le point de départ le moins cher qui garantisse la connexité.

Sans elle. On n'aurait aucune structure de base, et surtout on perdrait la borne $MST \leq OPT$ qui est le premier pilier de la preuve du facteur 1,5.

Étape 2 — Calcul des degrés

Pourquoi. Pour identifier ce qui empêche un parcours eulérien.

```
degree = [0] * len(cities)
for u, v, _ in mst_edges:
    degree[u] += 1
    degree[v] += 1
odd_vertices = [v for v, deg in enumerate(degree) if deg % 2 == 1]
```

Sans elle. On ne saurait pas quoi réparer.

Étape 3 — Identification des sommets impairs

Résultat chez nous : 12 sommets — Lyon, Nice, Strasbourg, Bordeaux, Lille, Reims, Saint-Étienne, Grenoble, Dijon, Angers, Nîmes, Clermont-Ferrand.

Pourquoi c'est un nombre pair. Lemme des poignées de main (§2.12). Ce n'est pas un hasard de notre instance : c'est un théorème.

Étape 4 — Couplage parfait de poids minimal

Pourquoi. Chaque sommet impair doit gagner exactement une arête pour devenir pair. Un couplage parfait donne exactement une arête à chacun, ni plus ni moins.

Sans elle. Les degrés resteraient impairs, aucun circuit eulérien n'existerait, et toute la suite s'effondrerait.

Pourquoi minimal. (a) Pratique : ces arêtes s'ajoutent au coût. (b) Théorique : la minimalité est ce qui permet de prouver $\text{couplage} \leq \text{OPT}/2$.

Notre code :

```
odd_graph = nx.Graph()
odd_graph.add_nodes_from(odd_vertices)
for idx, u in enumerate(odd_vertices):
    for v in odd_vertices[idx + 1:]:
        odd_graph.add_edge(u, v, weight=matrix[u][v])

matching = nx.algorithms.matching.min_weight_matching(odd_graph, weight="weight")
```

On construit le **sous-graphe complet induit** par les 12 sommets impairs — soit $C(12,2) = 66$ arêtes — puis on délègue le couplage à NetworkX.

Pourquoi déléguer ? Le couplage parfait de poids minimal se résout par l'algorithme des **fleurs (blossom)** d'Edmonds, en $O(n^3)$. C'est de très loin la brique la plus complexe de Christofides, et la réimplémenter n'aurait rien apporté à la compréhension du sujet — qui porte sur la modélisation et la comparaison, pas sur l'optimisation combinatoire avancée. Le sujet cite d'ailleurs NetworkX dans sa base de connaissances.

Précision importante à connaître. `min_weight_matching` de NetworkX calcule un couplage **de cardinalité maximale** et de poids minimal. Sur un graphe complet à nombre pair de sommets, **cardinalité maximale = couplage parfait**. Nous l'avons vérifié : le résultat contient bien 6 arêtes et couvre les 12 sommets impairs.

Étape 5 — Union MST u couplage

```
adjacency: dict[int, list[tuple[int, int]]] = defaultdict(list)
all_edges = mst_edges + matching_edges
for edge_id, (u, v, _) in enumerate(all_edges):
    adjacency[u].append((v, edge_id))
    adjacency[v].append((u, edge_id))
```

Résultat. $19 + 6 = 25$ **arêtes**, dans un **multigraphe** (une arête du couplage peut dupliquer une arête du MST).

Pourquoi tous les degrés sont pairs maintenant.

- Un sommet **pair** dans le MST n'est pas dans `odd_vertices`, donc le couplage ne le touche pas : il reste pair.
- Un sommet **impair** dans le MST est dans exactement une arête du couplage : son degré augmente de 1, et `impair + 1 = pair`.

Le graphe reste-t-il connexe ? Oui : le MST était déjà connexe, et on n'a fait qu'ajouter des arêtes.

Étape 6 — Circuit eulérien (Hierholzer)

Voir Partie 7.

Étape 7 — Shortcutting

Voir Partie 8.

Étape 8 — Cycle hamiltonien

Le résultat final : **3 445,60 km**.

6.3 Le code complet, commenté

```

def christofides(cities: list[City], start: int = 0) -> ChristofidesResult:
    matrix = distance_matrix(cities)
    graph = build_complete_graph(cities, matrix)

    # 1) Arbre couvrant minimal (Prim)
    mst_edges = prim_mst(graph, start=start)
    mst_weight = sum(weight for _, _, weight in mst_edges)

    # 2) Sommets de degré impair
    degree = [0] * len(cities)
    for u, v, _ in mst_edges:
        degree[u] += 1
        degree[v] += 1
    odd_vertices = [v for v, deg in enumerate(degree) if deg % 2 == 1]

    # 3) Couplage parfait de poids minimal sur ces sommets
    odd_graph = nx.Graph()
    odd_graph.add_nodes_from(odd_vertices)
    for idx, u in enumerate(odd_vertices):
        for v in odd_vertices[idx + 1:]:
            odd_graph.add_edge(u, v, weight=matrix[u][v])
    matching = nx.algorithms.matching.min_weight_matching(odd_graph, weight="weight")
    matching_edges = [(int(u), int(v), matrix[int(u)][int(v)]) for u, v in matching]
    matching_weight = sum(weight for _, _, weight in matching_edges)

    # 4) Union -> multigraphe eulérien
    adjacency = defaultdict(list)
    all_edges = mst_edges + matching_edges
    for edge_id, (u, v, _) in enumerate(all_edges):
        adjacency[u].append((v, edge_id))
        adjacency[v].append((u, edge_id))

    # 5) Circuit eulérien, puis raccourcis
    euler = _hierholzer_multigraph(adjacency, start)
    seen: set[int] = set()
    hamiltonian: list[int] = []
    for vertex in euler:
        if vertex not in seen:
            hamiltonian.append(vertex)
            seen.add(vertex)
    hamiltonian.append(hamiltonian[0]) # fermeture du cycle

    return ChristofidesResult(
        route=hamiltonian,
        distance_km=route_distance(hamiltonian, matrix),
        mst_weight_km=mst_weight,
        matching_weight_km=matching_weight,
        odd_vertices=odd_vertices,
    )

```

6.4 Pourquoi Christofides est pertinent pour Théobald

Le sujet demande explicitement de **justifier la pertinence** de Christofides. Quatre arguments :

1. **Nos distances sont métriques** (vérifié : zéro violation de l'inégalité triangulaire sur les 6 840 triplets). C'est précisément l'hypothèse sous laquelle Christofides fonctionne.
2. **Il offre une garantie**, ce qu'aucune heuristique classique (plus proche voisin, insertion) ne fait. Théobald sait qu'il ne fera jamais pire que $1,5 \times$ l'optimum.
3. **Il est déterministe et instantané** : quelques millisecondes, même résultat à chaque fois. Si Théobald doit recalculer sa tournée tous les matins, c'est idéal.
4. **Il sert de référence** : sans lui, on n'aurait aucun point de comparaison pour juger le génétique, ni la borne inférieure du MST.

PARTIE 7 — Hierholzer

7.1 Ce que fait l'algorithme

Il construit un **circuit eulérien** : un parcours fermé qui emprunte chaque arête exactement une fois.

Principe. On avance tant qu'on peut, en consommant les arêtes. Quand on est bloqué (plus d'arête disponible au sommet courant), on « ferme » ce bout de circuit et on revient en arrière chercher un sommet qui a encore des arêtes non utilisées. On fusionne les sous-circuits.

7.2 Notre implémentation

```
def _hierholzer_multigraph(adjacency, start):
    local = {node: list(edges) for node, edges in adjacency.items()} # copie
    used_edges: set[int] = set()
    stack = [start]
    circuit: list[int] = []

    while stack:
        v = stack[-1]
        while local[v] and local[v][-1][1] in used_edges: # purge des arêtes usées
            local[v].pop()
        if not local[v]: # plus rien à faire ici
            circuit.append(stack.pop()) # on ferme et on remonte
            continue
        u, edge_id = local[v].pop() # on prend une arête
        if edge_id in used_edges:
            continue
        used_edges.add(edge_id)
        stack.append(u) # on avance

    circuit.reverse()
    return circuit
```

Points à savoir expliquer :

- **local est une copie** (`list(edges)`), pour ne pas détruire le dictionnaire

d'adjacence d'origine. Sans cette copie, l'appelant se retrouverait avec une structure vidée.

- **used_edges est un ensemble d'identifiants d'arête**, pas de paires de sommets.

C'est ce qui permet de gérer les **arêtes parallèles** du multigraphe : deux arêtes entre les mêmes villes ont des `edge_id` différents, donc elles sont consommées séparément.

- **Chaque arête est marquée dès qu'on la prend d'un côté**, donc elle est

automatiquement ignorée quand on la rencontrera depuis l'autre extrémité.

- **Version itérative avec une pile**, pas récursive : on évite tout risque de

dépassement de pile et c'est plus facile à suivre.

- `circuit.reverse()` parce qu'on empile les sommets dans l'ordre de fermeture,

c'est-à-dire à l'envers.

7.3 Complexité

$O(|E|)$ — chaque arête est consommée une fois. Chez nous, 25 arêtes : instantané.

7.4 Pourquoi ce circuit n'est PAS la tournée

Parce qu'un circuit eulérien parle d'**arêtes**, pas de **sommets**. Certaines villes y apparaissent **plusieurs fois**.

Exemple concret. Dans notre multigraphe, un sommet du MST de degré 3 devient de degré 4 après le couplage. Le circuit eulérien doit donc y passer **deux fois** (4 arêtes = 2 entrées + 2 sorties).

Or Théobald ne veut pas repasser deux fois par la même ville : le sujet demande une visite unique. D'où l'étape suivante.

PARTIE 8 — Le shortcutting

8.1 L'idée

On parcourt le circuit eulérien du début à la fin. Dès qu'on rencontre une ville **déjà visitée**, on la **saute** : on relie directement la ville précédente à la suivante non encore visitée.

```
seen: set[int] = set()
hamiltonian: list[int] = []
for vertex in euler:
    if vertex not in seen:
        hamiltonian.append(vertex)
        seen.add(vertex)
    hamiltonian.append(hamiltonian[-1])
```

C'est trois lignes, mais c'est l'étape qui transforme Euler en Hamilton.

8.2 Pourquoi ça ne coûte rien

Sauter B, c'est remplacer le trajet $A \rightarrow B \rightarrow C$ par $A \rightarrow C$. Or l'inégalité triangulaire dit :

$$d(A, C) \leq d(A, B) + d(B, C)$$

Donc la tournée obtenue est **au plus aussi longue** que le circuit eulérien, jamais plus.

Généralisation. Si on saute plusieurs villes consécutives $B_1, B_2, \dots, B_k$, on applique l'inégalité triangulaire en cascade :

$$d(A, C) \leq d(A, B_1) + d(B_1, C) \leq d(A, B_1) + d(B_1, B_2) + d(B_2, C) \leq \dots$$

Le résultat tient toujours.

Chez nous. Le circuit eulérien pèse $\text{MST} + \text{couplage} = 2\,665,05 + 1\,142,03 = \mathbf{3\,807,08\ km}$, et après raccourcis la tournée ne fait plus que $\mathbf{3\,445,60\ km}$. Les raccourcis ont donc fait **gagner 361,48 km**. C'est une vérification numérique très convaincante à sortir à l'oral.

8.3 Ce que l'inégalité triangulaire ne garantit pas

Elle garantit que le raccourci ne **dégrade** pas. Elle ne garantit **pas** que le résultat soit optimal — on reste dans une approximation.

Elle ne garantit pas non plus que l'ordre choisi soit le meilleur : le circuit eulérien produit par Hierholzer n'est pas unique, et un autre circuit aurait pu donner une tournée différente (meilleure ou pire) après raccourcis.

Question probable. « *Christofides est-il déterministe si Hierholzer peut produire plusieurs circuits ?* »
→ « Oui, dans notre implémentation, parce que l'ordre d'exploration est entièrement déterminé par l'ordre des listes d'adjacence, lui-même déterminé par l'ordre de construction. Il n'y a aucun tirage aléatoire. Le résultat est donc reproductible à l'identique, ce que nos exécutions confirment. »

PARTIE 9 — D'où vient le facteur 1,5 ?

C'est la question la plus probable de tout l'oral. Voici la preuve, avec ses hypothèses.

9.1 Hypothèses nécessaires

1. Le graphe est **complet**.
2. Les distances forment une **métrique** — en particulier, l'inégalité triangulaire.

Sans (2), le théorème est **faux**. Sur un TSP général (non métrique), on sait qu'aucun algorithme d'approximation à facteur constant n'existe, sauf si $P = NP$.

9.2 Notations

- OPT = longueur de la tournée optimale.
- M = poids de l'arbre couvrant minimal.
- W = poids du couplage parfait de poids minimal sur les sommets impairs.

9.3 Premier pilier : $M \leq OPT$

Démontré en §4.4 : retirer une arête à la tournée optimale donne un arbre couvrant, donc de poids $\geq M$.

9.4 Deuxième pilier : $W \leq OPT / 2$

C'est la partie la plus subtile. Voici l'argument.

1. Soit O l'ensemble des sommets de degré impair. $|O|$ est pair (lemme des poignées de main) — disons $|O| = 2k$. Chez nous, $2k = 12$.
2. Considérons la tournée optimale T^* . Parcourez-la et notez **uniquement** les sommets de O dans l'ordre où vous les rencontrez. Vous obtenez un cycle C sur les sommets de O seulement.
3. **Ce cycle C coûte au plus OPT .** Pourquoi ? Parce que chaque arête de C relie deux sommets de O consécutifs le long de T^* , en court-circuitant les sommets pairs intermédiaires. Par inégalité triangulaire, chaque raccourci est au plus égal au morceau de T^* qu'il remplace. Donc $poids(C) \leq OPT$.
4. C est un cycle de longueur paire ($2k$ sommets). **On peut donc le décomposer en deux couplages parfaits disjoints A et B** : il suffit de prendre une arête sur deux. (C'est possible précisément parce que $2k$ est pair.)
5. $poids(A) + poids(B) = poids(C) \leq OPT$. Donc le **moins cher** des deux vérifie $\min(poids(A), poids(B)) \leq OPT / 2$.
6. Or A et B sont des couplages parfaits sur O . Comme W est le couplage parfait **de poids minimal** sur O , il est au moins aussi bon :

$$W \leq \min(poids(A), poids(B)) \leq OPT / 2$$

C'est ici que la **minimalité** du couplage sert : sans elle, l'étape 6 est fausse.

9.5 Troisième pilier : le raccourci ne coûte rien

Le multigraphe eulérien pèse $M + W$. Le circuit eulérien pèse exactement $M + W$ (il emprunte chaque arête une fois). Le shortcutting ne peut qu'abaisser ce coût (§8.2).

9.6 Conclusion

$$\text{poids(tournée Christofides)} \leq M + W \leq \text{OPT} + \text{OPT}/2 = 1,5 \times \text{OPT}$$

■

9.7 Ce que ça veut dire, et ce que ça ne veut PAS dire

☐ Vrai	☐ Faux
La tournée ne dépasse jamais $1,5 \times \text{OPT}$.	La tournée vaut $1,5 \times \text{OPT}$.
C'est une borne supérieure garantie dans le pire cas.	C'est une estimation de la qualité typique.
Vrai uniquement pour un TSP métrique .	Vrai pour tout TSP.
En pratique, Christofides tourne autour de 10 % au-dessus de l'optimum.	Christofides est le meilleur algorithme possible.

Question piège. « Votre Christofides fait 3 445,60 km. L'optimum vaut-il donc 2 297 km ? » → « Non. $3\,445,60 / 1,5 = 2\,297,07$ km est seulement une borne **inférieure** déduite de la garantie. Et elle est moins bonne que celle du MST, qui nous donne $\text{OPT} \geq 2\,665,05$ km. Donc la garantie de Christofides, ici, ne nous apprend rien de plus que le MST sur la localisation de l'optimum. »

Question de niveau supérieur. « Le facteur 1,5 a-t-il été amélioré ? » → « Oui, très récemment. Le facteur 1,5 de Christofides (1976) est resté le meilleur connu pendant plus de 40 ans. En 2020, Karlin, Klein et Oveis Gharan ont publié un algorithme en $1,5 - \epsilon$ pour un ϵ de l'ordre de 10^{-36} . C'est une percée théorique majeure, mais sans aucune portée pratique. »

9.8 La qualité réelle de notre Christofides

Nous pouvons **encadrer** son écart à l'optimum sans connaître l'optimum :

$$\text{Christofides} / \text{OPT} \leq \text{Christofides} / \text{MST} = 3\,445,60 / 2\,665,05 = 1,293$$

Donc notre tournée Christofides est **au plus 29,3 % au-dessus de l'optimum**, ce qui est nettement mieux que la garantie théorique de 50 %. Même raisonnement pour le génétique :

$$\text{GA} / \text{OPT} \leq 3\,157,13 / 2\,665,05 = 1,185 \rightarrow \text{au plus 18,5 \% au-dessus de l'optimum}$$

C'est une analyse que peu de groupes font, et elle impressionne.

PARTIE 10 — L'algorithme génétique : les concepts

10.1 Métaheuristique, stochastique, évolution

Heuristique. Une méthode de résolution qui cherche une bonne solution sans garantir l'optimum.

Métaheuristique. Un **schéma général** de recherche, applicable à de nombreux problèmes, qu'on instancie pour un problème donné. Les algorithmes génétiques, le recuit simulé, les colonies de fourmis en sont.

Stochastique. Qui fait intervenir le hasard. Deux exécutions avec des graines différentes peuvent donner des résultats différents. C'est le contraire de **déterministe**.

Évolution. La métaphore biologique : une population d'individus, une pression de sélection, de la reproduction avec recombinaison, et des mutations. Les individus les mieux adaptés (ici : les tournées les plus courtes) ont plus de chances de transmettre leurs caractéristiques.

⚠ **Attention à la métaphore.** L'algorithme génétique n'est pas une simulation biologique et ne « comprend » rien au TSP. C'est une méthode de recherche aléatoire guidée. Dites-le si on vous pousse sur le sujet : c'est un signe de maîtrise.

10.2 Le vocabulaire, relié à notre code

Terme	Définition	Dans <code>src/genetic.py</code>
Individu	une solution candidate	une <code>list[int]</code> de 19 indices
Chromosome	la représentation de l'individu	la liste elle-même
Gène	un élément du chromosome	un indice de ville, à une position donnée
Population	l'ensemble des individus d'une génération	<code>population: list[list[int]]</code> , 160 éléments
Fitness	la qualité d'un individu	<code>_closed_distance</code> — à minimiser
Sélection	choix des parents	<code>_tournament</code>
Croisement	production d'enfants	<code>_ordered_crossover</code>
Mutation	perturbation aléatoire	<code>_mutate</code>
Élitisme	conservation des meilleurs	<code>ranked[: config.elite_size]</code>
Génération	une itération complète	une passe de la boucle <code>for _ in range(config.generations)</code>
Convergence	stabilisation de la meilleure valeur	visible dans <code>history</code>

10.3 La boucle, ligne par ligne

```

rng = random.Random(config.seed)                # générateur isolé, reproductible
matrix = distance_matrix(cities)
genes = list(range(1, len(cities)))              # 1..19 : Paris (0) est exclu

# --- population initiale : 160 permutations aléatoires
population = []
for _ in range(config.population_size):
    chromosome = genes.copy()
    rng.shuffle(chromosome)
    population.append(chromosome)

best_individual, best_distance, history = None, float("inf"), []

for _ in range(config.generations):
    # 1. Évaluation
    distances = [_closed_distance(ind, matrix) for ind in population]
    ranked = sorted(range(len(population)), key=lambda idx: distances[idx])

    # 2. Mémorisation du meilleur de tous les temps
    if distances[ranked[0]] < best_distance:
        best_distance = distances[ranked[0]]
        best_individual = population[ranked[0]].copy()
    history.append(best_distance)

    # 3. Élitisme : les meilleurs passent tels quels
    new_population = [population[idx].copy() for idx in ranked[: config.elite_size]]

    # 4. Reproduction jusqu'à remplir la population
    while len(new_population) < config.population_size:
        parent1 = _tournament(population, distances, rng, config.tournament_size)
        parent2 = _tournament(population, distances, rng, config.tournament_size)

        if rng.random() < config.crossover_rate:
            child1, child2 = _ordered_crossover(parent1, parent2, rng)
        else:
            child1, child2 = parent1.copy(), parent2.copy()

        if rng.random() < config.mutation_rate:
            _mutate(child1, rng)
        if rng.random() < config.mutation_rate:
            _mutate(child2, rng)

        new_population.append(child1)
        if len(new_population) < config.population_size:
            new_population.append(child2)

    population = new_population

route = [0] + best_individual + [0]

```

Détails à savoir défendre :

- `random.Random(config.seed)` crée un générateur **local**. On ne touche pas au

générateur global de Python : deux exécutions ne peuvent pas s'influencer, et le résultat ne dépend pas de ce qui a été tiré ailleurs dans le programme.

- `.copy()` partout où un individu passe d'une génération à l'autre : sans ça, deux

entrées de la population pointeraient sur la **même liste**, et une mutation sur l'une modifierait l'autre. C'est le bug classique du GA en Python.

- `best_individual` **est mémorisé séparément** de la population. Même si un accident

faisait disparaître le meilleur, on le garde. C'est une forme d'élitisme externe, qui s'ajoute à l'élitisme interne.

- **history** enregistre le meilleur *de tous les temps*, pas le meilleur de la

génération courante. C'est pour cela que la courbe de convergence est **monotone décroissante** : elle ne remonte jamais.

- **La dernière ligne referme le cycle** : `[0] + best + [0]`, départ et retour à Paris.

PARTIE 11 — La représentation : pourquoi une permutation

11.1 Une tournée est une permutation

Visiter 20 villes une fois chacune, c'est choisir un **ordre**. Un ordre sur un ensemble fini, c'est une permutation. D'où la représentation naturelle : une liste sans doublon contenant chaque ville exactement une fois.

11.2 Pourquoi Paris est fixé

```
genes = list(range(1, len(cities))) # Fix Paris (index 0) as anchor.
```

Paris est l'indice 0 dans le CSV (première ligne). Le chromosome contient donc uniquement les indices 1 à 19 — **19 gènes**, pas 20.

La raison. Un cycle est invariant par décalage :

```
Paris → Lyon → Marseille → Paris
Lyon  → Marseille → Paris → Lyon
Marseille → Paris → Lyon → Marseille
```

Ces trois écritures désignent **le même cycle**, avec la même longueur. Si le chromosome contenait les 20 villes, l'algorithme gaspillerait de la population à explorer 20 écritures différentes de la même solution.

En fixant une ville d'ancrage :

- l'espace de recherche passe de **20!** à **19!** représentations — on divise par 20 ;
- **aucune solution n'est perdue** : tout cycle peut s'écrire en commençant par Paris ;
- le croisement et la mutation deviennent plus efficaces, parce que deux chromosomes

différents désignent vraiment deux tournées différentes.

Question probable. « *Pourquoi Paris, et pas une autre ville ?* » → « Le choix est arbitraire : n'importe quelle ville ancre aurait le même effet. Paris est simplement la première ligne du CSV, donc l'indice 0. Cela colle aussi à l'histoire : Théobald part de chez lui et y revient. »

Question probable. « *Cela force-t-il Théobald à partir de Paris ?* » → « Ça ne change rien à la tournée : le cycle est le même quel que soit le point d'entrée. Si Théobald habitait Lyon, il parcourrait exactement le même circuit, simplement en le commençant à Lyon. La distance serait identique. »

11.3 Le sens de parcours

Nous **n'avons pas** éliminé la symétrie de sens. Le chromosome `[Lyon, Nice, ...]` et son miroir `[..., Nice, Lyon]` représentent le même cycle et ont la même fitness, mais comptent comme deux individus distincts.

Pourquoi c'est acceptable. L'éliminer imposerait une contrainte supplémentaire (forcer par exemple `chromosome[0] < chromosome[-1]`) qui compliquerait le croisement sans gain mesuré. C'est une amélioration possible à mentionner, pas un défaut grave.

À dire si on vous le reproche : « C'est exact, nous n'avons éliminé qu'une des deux symétries. L'espace de représentation reste donc deux fois plus grand que l'espace des solutions réellement distinctes. C'est une optimisation que nous n'avons pas jugée prioritaire, mais qui serait facile à ajouter. »

PARTIE 12 — La fitness

12.1 Le calcul

```
def _closed_distance(individual: list[int], matrix: list[list[float]]) -> float:
    # City 0 is omitted from the chromosome and fixed as the cycle anchor.
    route = [0] + individual + [0]
    return route_distance(route, matrix)

def route_distance(route, matrix) -> float:
    route_list = list(route)
    if len(route_list) < 2:
        return 0.0
    return sum(matrix[route_list[i]][route_list[i + 1]]
                for i in range(len(route_list) - 1))
```

`route_distance` somme les distances entre sommets **consécutifs** de la liste qu'on lui donne. Elle ne referme **pas** le cycle toute seule — c'est le rôle de `_closed_distance` d'ajouter le 0 au début et à la fin.

Conséquence. Pour un chromosome de 19 gènes, la route fait 21 éléments et la somme comporte **20 termes** : les 20 trajets du circuit fermé.

12.2 Pourquoi inclure le retour au départ

Parce que le sujet le demande : « *revenir au point de départ* ». Sans ce dernier segment, l'algorithme optimiserait un **chemin ouvert**, et il aurait tendance à terminer dans une ville très éloignée de Paris — ce qui donnerait une tournée réelle bien plus longue une fois le retour ajouté.

Question piège. « *Que se passerait-il si vous oubliiez le retour ?* » → « On résoudrait un problème différent — le *path TSP* au lieu du *cycle TSP*. Le GA trouverait un chemin qui finit loin de Paris, et la vraie tournée serait plus longue. C'est une erreur facile à commettre, c'est pour ça que `_closed_distance` est une fonction à part avec un commentaire explicite. »

12.3 Minimiser ou maximiser ?

Dans la littérature, la « fitness » désigne souvent une quantité à **maximiser** (plus un individu est adapté, plus sa fitness est haute). Ici, notre fonction objectif est une **distance**, donc elle est à **minimiser**.

Notre convention. Nous travaillons directement sur la distance, et partout où il faut « le meilleur », nous prenons le **minimum** :

```
best = min(indices, key=lambda idx: distances[idx])      # tournoi
ranked = sorted(range(len(population)), key=lambda idx: distances[idx]) # tri croissant
```

Une **alternative classique** serait de définir $\text{fitness} = 1 / \text{distance}$ ou $\text{fitness} = C - \text{distance}$ pour retomber sur une maximisation. C'est indispensable si on utilise une sélection par **roulette** (qui a besoin de probabilités proportionnelles à la fitness). Avec une sélection par **tournoi**, qui ne fait que **comparer** deux valeurs, aucune transformation n'est nécessaire.

C'est un excellent argument à sortir : « nous n'avons pas eu besoin de transformer la fitness précisément parce que le tournoi ne compare que des ordres, pas des proportions. »

PARTIE 13 — La sélection par tournoi

13.1 Le code

```
def _tournament(population, distances, rng, k):
    indices = rng.sample(range(len(population)), k=min(k, len(population)))
    best = min(indices, key=lambda idx: distances[idx])
    return population[best]
```

13.2 Le principe

1. Tirer **k individus au hasard** dans la population (sans remise — `rng.sample`).
2. Retenir le meilleur des k.
3. Il devient parent.

Chez nous, $k = 5$ dans la configuration retenue.

Exemple. On tire 5 tournées de 3 412, 3 289, 3 877, 3 157 et 3 602 km. Le parent est celle de 3 157 km.

13.3 La pression de sélection

Définition. L'intensité avec laquelle l'algorithme favorise les bons individus au détriment des autres.

k	Effet
k = 1	tirage purement aléatoire, aucune pression : la recherche devient une marche au hasard, l'algorithme ne progresse pas.
k petit (2-3)	pression douce, beaucoup de diversité, convergence lente.
k = 5 (notre choix)	compromis. Un individu médiocre garde une chance réelle d'être parent.
k grand (20+)	pression très forte : seuls les tout meilleurs se reproduisent, la population s'uniformise vite et se bloque dans un optimum local.
k = taille de la population	le meilleur individu est toujours choisi : plus aucune diversité.

Formulation à retenir : « *k règle l'équilibre entre exploitation (favoriser ce qui marche déjà) et exploration (laisser une chance à autre chose).* »

13.4 Tournoi vs roulette

	Tournoi	Roulette (roue biaisée)
Principe	comparer k individus, garder le meilleur	probabilité proportionnelle à la fitness

	Tournoi	Roulette (roue biaisée)
Besoin d'une fitness positive à maximiser	non	oui
Sensible à l'échelle des valeurs	non	oui , fortement
Risque de domination précoce	faible	élevé si un individu est très supérieur
Réglage	un entier k , très lisible	nécessite souvent un <i>scaling</i>
Coût	0(k)	0(n) ou tri préalable

Pourquoi le tournoi chez nous. Nos valeurs de fitness sont des distances comprises entre $\approx 3\,100$ et $\approx 6\,500$ km : elles sont toutes du même ordre de grandeur. Une roulette sur $1/\text{distance}$ donnerait des probabilités quasiment identiques pour tous les individus — la pression de sélection serait ridiculement faible. Il faudrait ajouter une mise à l'échelle (rank scaling, sigma scaling...), c'est-à-dire un paramètre de plus à régler. Le tournoi évite tout cela : il ne regarde que l'**ordre**, pas les valeurs.

13.5 Autres sélections existantes

- **Sélection par rang** : probabilité fonction du rang, pas de la valeur. Proche du tournoi dans l'esprit.
- **Sélection par troncature** : on ne garde que les $x\%$ meilleurs. Pression très forte.
- **SUS (Stochastic Universal Sampling)** : variante de la roulette qui réduit la variance d'échantillonnage.

PARTIE 14 — Le croisement OX (Ordered Crossover)

14.1 Pourquoi un croisement « normal » ne marche pas

Un croisement à un point classique coupe les deux parents et recolle les morceaux :

```

Parent 1 : B C | D E F G H
Parent 2 : D G | B H C F E
          ↓
Enfant   : B C | B H C F E

```

Résultat : **B et C apparaissent deux fois, D, E, G, H manquent**. Ce n'est plus une permutation, donc plus une tournée valide. L'individu est inutilisable.

*C'est **la** difficulté des algorithmes génétiques appliqués au TSP, et c'est exactement ce que le professeur veut vous entendre expliquer.*

14.2 Comment OX fonctionne

1. Choisir deux points de coupe au hasard, définissant un **segment**.
2. **Copier ce segment du parent 1** dans l'enfant, à la même position.
3. Parcourir le **parent 2** dans l'ordre et **remplir les cases vides** de l'enfant avec les villes qui n'y sont pas encore.

14.3 Exemple complet

```
Parent 1 : B C [D E F] G H
Parent 2 : D G B H C F E
segment = positions 2 à 4 → D E F
```

Étape 1 — on copie le segment du parent 1 :
Enfant : _ _ [D E F] _ _

Étape 2 — on lit le parent 2 dans l'ordre : D, G, B, H, C, F, E
on retire ceux déjà présents (D, E, F) → il reste : G, B, H, C

Étape 3 — on remplit les trous de gauche à droite :
Enfant : G B [D E F] H C

Résultat : **G B D E F H C** — chaque ville exactement une fois. **Permutation valide.**

14.4 Notre implémentation

```
def _ordered_crossover(a, b, rng):
    n = len(a)
    left, right = sorted(rng.sample(range(n), 2))

    def make_child(p1, p2):
        child = [-1] * n
        child[left:right] = p1[left:right]
        remaining = [gene for gene in p2 if gene not in child]
        it = iter(remaining)
        for i in list(range(0, left)) + list(range(right, n)):
            child[i] = next(it)
        return child

    return make_child(a, b), make_child(b, a)
```

Points à savoir défendre :

- `child = [-1] * n` : `-1` ne peut jamais être un indice de ville valide, donc il sert de marqueur « case vide » sans ambiguïté.

- `gene not in child` : on filtre les villes déjà placées. C'est $O(n)$ par test, donc $O(n^2)$ au total — acceptable pour $n = 19$, et bien plus lisible qu'un `set`.

- **Deux enfants par croisement** : `make_child(a, b)` et `make_child(b, a)`. On exploite mieux le couple de parents et on remplit la population deux fois plus vite.

- `sorted(rng.sample(range(n), 2))` garantit `left < right`. Notez que `right`

vaut au maximum $n-1$, donc `child[left:right]` n'inclut jamais le tout dernier gène dans le segment conservé. C'est un biais mineur de l'implémentation, sans effet mesuré — mais sachez le reconnaître si on vous le montre.

14.5 Pourquoi OX préserve la permutation

Par construction :

- le segment copié contient des villes **distinctes** (il vient d'une permutation) ;
- `remaining` ne contient **que** des villes absentes du segment ;
- on place exactement autant d'éléments qu'il y a de cases vides.

Donc chaque ville apparaît **une et une seule fois**. La validité est garantie, pas seulement probable — il n'y a jamais besoin de « réparer » l'enfant.

14.6 Ce que OX transmet

OX conserve l'**ordre relatif** des villes du parent 2 et une **sous-séquence contiguë** du parent 1. Pour le TSP, l'information utile est justement l'ordre de visite — un bon enchaînement local (« Marseille puis Toulon puis Nice ») a des chances d'être transmis. C'est ce qui rend OX pertinent ici.

14.7 Les alternatives

Croisement	Principe	Pourquoi pas ici
PMX (Partially Mapped)	échange un segment puis répare par une table de correspondance	Équivalent en qualité, plus complexe à expliquer.
CX (Cycle Crossover)	conserve la position absolue des villes	Préserve les positions plutôt que l'ordre : moins pertinent pour un cycle.
ERX (Edge Recombination)	travaille sur les arêtes et non les positions	Souvent le meilleur pour le TSP, mais nettement plus lourd à implémenter.
OX (notre choix)	segment + ordre relatif	Simple, correct par construction, adapté à l'ordre.

Réponse honnête : « ERX est réputé meilleur pour le TSP parce qu'il raisonne en arêtes, ce qui correspond exactement à ce qui coûte. Nous avons choisi OX pour sa simplicité et parce qu'il garantit la validité sans réparation. Tester ERX serait une amélioration intéressante. »

PARTIE 15 — La mutation

15.1 Le code

```
def _mutate(individual: list[int], rng: random.Random) -> None:
    """Mix swap and inversion mutations to preserve permutation validity."""
    if len(individual) < 2:
        return
    i, j = sorted(rng.sample(range(len(individual)), 2))
    if rng.random() < 0.5:
        individual[i], individual[j] = individual[j], individual[i]    # swap
    else:
        individual[i : j + 1] = reversed(individual[i : j + 1])        # inversion
```

Notez : la fonction modifie l'individu **en place** et ne retourne rien.

15.2 Les deux opérateurs

Échange (swap)

On échange deux villes de position.

```
A [B] C D [E] F  →  A [E] C D [B] F
```

Effet : modifie **4 arêtes** de la tournée. C'est une perturbation assez brutale et souvent peu utile pour le TSP.

Inversion de segment (2-opt move)

On retourne un segment entier.

```
A [B C D E] F → A [E D C B] F
```

Effet : ne modifie que **2 arêtes** — celles aux extrémités du segment. Les arêtes internes sont conservées (simplement parcourues à l'envers, ce qui ne change rien puisque le graphe est non orienté).

C'est exactement le mouvement de base de l'algorithme 2-opt. C'est un opérateur reconnu comme très efficace sur le TSP, parce qu'il « décroise » deux arêtes qui se chevauchent. Si vous ne deviez retenir qu'une chose : **l'inversion est l'opérateur utile, le swap est le complément aléatoire.**

Notre mélange 50/50 donne à l'algorithme deux types de perturbation : une fine et géométriquement pertinente (inversion), une plus désordonnée (swap) qui aide à sortir des blocages.

15.3 Les deux préservent la permutation

Échanger deux éléments ou renverser un segment ne fait que **déplacer** des gènes : rien n'est ajouté, rien n'est supprimé. La validité est donc automatique.

15.4 Pourquoi la mutation est indispensable

Sans mutation (taux = 0 %). Le croisement ne fait que **recombinaison** ce qui existe déjà. Si une ville n'apparaît jamais à une certaine position dans toute la population, aucun croisement ne l'y mettra. La population **converge prématurément** vers un optimum local et n'en sort plus jamais. C'est un danger réel et bien documenté.

Avec une mutation trop forte (taux ≈ 100 %). Chaque enfant est systématiquement perturbé. On détruit en permanence les bonnes combinaisons trouvées par le croisement : l'algorithme dégénère en **recherche aléatoire**. Il ne converge plus.

Notre taux : 22 %. Environ un enfant sur cinq est muté.

⚠ **Formulation critique.** Ne dites **jamais** « 22 % est le taux optimal ». Dites : « 22 % est la valeur de notre configuration équilibrée, qui s'est montrée la plus stable sur nos trois seeds lors du benchmark. Ce n'est pas une constante universelle : la littérature recommande souvent des taux plus faibles, de l'ordre de 1 à 5 %, mais avec des opérateurs et des tailles de population différents. »

15.5 Détail : mutation par enfant, pas par gène

Dans notre code, `mutation_rate` est la probabilité qu'un **enfant entier** subisse **une** mutation :

```
if rng.random() < config.mutation_rate:
    _mutate(child1, rng)
```

Une autre convention courante applique le taux **gène par gène** (chaque position a une probabilité p d'être perturbée). Avec cette convention, 22 % serait énorme. C'est pourquoi les taux de la littérature ne sont pas directement comparables aux nôtres — un point à savoir si on vous objecte que « 22 % c'est beaucoup trop ».

PARTIE 16 — L'élitisme

16.1 Le code

```
new_population = [population[idx].copy() for idx in ranked[: config.elite_size]]
```

Les `elite_size` meilleurs individus (ici **6** sur 160) sont copiés tels quels dans la génération suivante, **sans croisement ni mutation**.

16.2 L'avantage

Garantie de non-régression. Sans élitisme, il est parfaitement possible que la meilleure tournée d'une génération soit détruite par un croisement ou une mutation, et que la génération suivante soit globalement moins bonne. L'élitisme rend la suite des meilleures valeurs **monotone décroissante**.

C'est ce qu'on voit sur notre courbe de convergence : elle ne remonte jamais.

16.3 Le risque

Perte de diversité. Si l'élite est trop grosse, elle sature la population de copies quasi identiques. La recherche s'enferme autour d'une seule solution et l'exploration meurt.

Ordre de grandeur raisonnable : 1 à 10 % de la population. Chez nous : $6 / 160 = 3,75 \%$. C'est modéré et sain.

16.4 Élitisme externe

Notre code ajoute une seconde sécurité : `best_individual` est mémorisé **hors de la population**. Même si l'élite venait à disparaître, la meilleure solution jamais vue est conservée et c'est elle qui est renvoyée :

```
route = [0] + best_individual + [0]
```

Question probable. « *L'élitisme ne fausse-t-il pas l'évolution ?* » → « Il la biaise volontairement, oui. En biologie, aucun individu n'est immortel. Mais nous ne faisons pas de la biologie : nous optimisons. Garantir qu'on ne perd jamais la meilleure solution trouvée est un avantage pur en optimisation. »

PARTIE 17 — Exploration vs exploitation

17.1 Le dilemme

C'est **le** concept central de toutes les métaheuristiques.

- **Exploiter** : approfondir ce qui marche déjà, affiner les bonnes solutions.

Trop d'exploitation → on se fige dans un optimum local.

- **Explorer** : essayer des choses nouvelles, aller voir ailleurs.

Trop d'exploration → on ne converge jamais, c'est du hasard pur.

Un bon algorithme génétique **équilibre** les deux.

17.2 Quel paramètre agit dans quel sens

Paramètre	Augmenter pousse vers...	Effet
Taille de population	exploration	plus de diversité initiale, plus de chemins explorés en parallèle ; coût linéaire en temps
Taille du tournoi (k)	exploitation	pression plus forte, convergence plus rapide mais plus risquée
Taux de mutation	exploration	échappe aux optima locaux ; trop haut, détruit l'acquis
Taux de croisement	exploitation	recombine l'existant ; à 100 %, aucun individu ne passe intact
Taille de l'élite	exploitation	sécurise, mais uniformise si trop grande
Nombre de générations	les deux	laisse plus de temps ; n'aide plus une fois convergé

17.3 Nos trois configurations vues sous cet angle

Config	Pop.	Gén.	k	Mut.	Élite	Positionnement
rapide	80	220	4	18 %	4	peu de tout : convergence rapide mais fragile
équilibrée	160	520	5	22 %	6	compromis — retenue
exploratoire	240	800	6	30 %	8	forte diversité, mais aussi k plus élevé

Observation à savoir commenter. La configuration « exploratoire » augmente la mutation (→ exploration) **mais aussi** la taille du tournoi (→ exploitation). Ces deux réglages tirent en sens opposés, ce qui explique en partie qu'elle ne surpasse pas l'équilibrée malgré 3,6 fois plus de calcul.

C'est une analyse honnête et fine : assumez-la. « Avec le recul, notre configuration exploratoire n'est pas purement exploratoire, puisqu'elle augmente aussi la pression de sélection. C'est une limite de notre plan d'expérience. »

PARTIE 18 — Le seed (graine aléatoire)

18.1 Ce que c'est vraiment

Un ordinateur ne produit pas de vrai hasard. Il utilise un **générateur pseudo-aléatoire** (PRNG) : un algorithme **déterministe** qui, à partir d'une valeur initiale — la **graine** (seed) — produit une suite de nombres qui *ressemble* à du hasard.

Propriété fondamentale : même graine → **exactement** la même suite de nombres → exactement le même résultat.

```
rng = random.Random(config.seed)
```

Python utilise le Mersenne Twister (MT19937).

18.2 Pourquoi nous fixons le seed

Reproductibilité. Le sujet demande un dépôt GitHub public. N'importe qui doit pouvoir relancer `python main.py` et retrouver **3 157,13 km**. Sans seed fixé, chaque exécution donnerait un résultat différent et

le README serait invérifiable.

Nous l'avons vérifié : les distances sont identiques au chiffre près entre nos exécutions, seuls les **temps** varient.

18.3 Pourquoi tester plusieurs seeds

Un algorithme stochastique peut avoir de la chance. Une seule exécution ne dit strictement **rien** sur sa fiabilité.

Notre preuve par l'exemple. Avec la configuration « rapide » :

Seed	Distance obtenue
11	3 334,21 km
22	3 157,13 km
33	3 157,13 km

Si nous n'avions testé que le seed 22, nous aurions conclu que la configuration rapide est excellente. Si nous n'avions testé que le seed 11, nous aurions conclu qu'elle est médiocre. **Les deux conclusions auraient été fausses.**

Même chose pour la configuration « exploratoire » :

Seed	Distance obtenue
11	3 157,13 km
22	3 334,21 km
33	3 157,13 km

Découverte à raconter absolument. Les configurations « rapide » et « exploratoire » ont des statistiques **rigoureusement identiques** (best 3 157,13 ; moyenne 3 216,15 ; écart-type 83,48 ; pire 3 334,21). Ce n'est pas une coïncidence suspecte : c'est parce que **les deux tombent sur les deux mêmes attracteurs** — 3 157,13 km et 3 334,21 km — simplement sur des seeds différents. Notre instance possède visiblement deux optima locaux très marqués, et c'est une observation intéressante en soi.

18.4 Le seed du résultat final

`main.py` relance la configuration retenue sur les trois seeds et garde **le meilleur** :

```
ga, ga_best_time = min(ga_runs, key=lambda item: item[0].distance_km)
```

Le `summary.json` indique `"seed": 11` pour la tournée finale.

Question piège. « Vous gardez le meilleur des 3 : n'est-ce pas de la triche ? » → « Non, mais il faut le dire honnêtement. Garder le meilleur de plusieurs exécutions est la pratique standard avec une métaheuristique : on veut la meilleure tournée pour Théobald. En revanche, ce résultat ne caractérise pas la performance *typique* de l'algorithme — c'est précisément le rôle du benchmark, qui publie la moyenne, le pire et l'écart-type. Nous publions les deux. »

PARTIE 19 — Le benchmark et ses statistiques

19.1 Le code

```
def benchmark_configs(cities, configs, seeds):
    rows = []
    for name, base in configs.items():
        values = []
        for seed in seeds:
            config = GAConfig(..., seed=seed)
            result = genetic_tsp(cities, config)
            values.append(result.distance_km)
        rows.append({
            "configuration": name,
            "runs": len(values),
            "best_km": min(values),
            "mean_km": mean(values),
            "std_km": pstdev(values),
            "worst_km": max(values),
        })
    return rows
```

9 exécutions au total : 3 configurations × 3 seeds.

19.2 Les résultats réels

Configuration	Runs	Meilleure	Moyenne	Écart-type	Pire
rapide	3	3 157,13	3 216,15	83,48	3 334,21
équilibrée	3	3 157,13	3 157,13	0,00	3 157,13
exploratoire	3	3 157,13	3 216,15	83,48	3 334,21

19.3 Les statistiques, expliquées

Meilleure (best). Le minimum des 3 valeurs. Répond à : « *de quoi cet algorithme est-il capable au mieux ?* »

Moyenne (mean). $(v_1 + v_2 + v_3) / 3$. Répond à : « *à quoi puis-je m'attendre en moyenne ?* »

Pire (worst). Le maximum. Répond à : « *quel est le risque ?* »

Écart-type (standard deviation). Mesure la **dispersion** autour de la moyenne.

- $\sigma = 0$ → toutes les exécutions donnent exactement le même résultat.
- σ grand → résultats très variables, algorithme peu fiable.

Variance. Le carré de l'écart-type. Même information, mais pas dans l'unité des données (des km², ici) — c'est pourquoi on préfère l'écart-type.

Détail technique à connaître. Nous utilisons `statistics.pstdev`, l'écart-type **de population** (division par n). `statistics.stdev` serait l'écart-type **d'échantillon** (division par $n-1$), utilisé quand on veut estimer la dispersion d'une population plus large à partir d'un échantillon. Avec seulement 3 valeurs, le choix change sensiblement le chiffre (83,48 contre 102,24), mais pas le classement des configurations. Nous traitons nos 3 seeds comme la population complète de nos essais, d'où `pstdev`.

19.4 Pourquoi ne regarder que « best » serait trompeur

Les **trois** configurations ont exactement le même **best** : 3 157,13 km. Sur ce seul critère, elles seraient **indiscernables** et on ne pourrait rien conclure.

C'est la **moyenne** et surtout l'**écart-type** qui les départagent : l'équilibrée atteint ce résultat **à chaque fois**, les deux autres seulement 2 fois sur 3.

Analogie à sortir à l'oral : « Si vous choisissiez un livreur, vous ne le choisiriez pas sur sa meilleure livraison de l'année, mais sur sa régularité. »

19.5 Comment la configuration est choisie par le code

```
best_config_name = min(
    benchmark,
    key=lambda row: (float(row["mean_km"]), float(row["std_km"]), float(row["best_km"]))
)['configuration']
```

Tri **lexicographique** sur un triplet :

1. d'abord la **moyenne** la plus faible ;
2. en cas d'égalité, l'**écart-type** le plus faible ;
3. en cas d'égalité encore, la **meilleure** valeur.

Ici, l'équilibrée gagne dès le premier critère (3 157,13 < 3 216,15).

Point à valoriser : le choix n'est pas fait à la main, il est **calculé par le programme** selon un critère explicite et vérifiable. C'est une démarche reproductible, pas une préférence subjective.

19.6 Les limites honnêtes de notre benchmark

1. **3 seeds, c'est très peu.** Aucun test statistique sérieux ne se fonde sur 3 observations. Un écart-type de 0 sur 3 tirages ne prouve pas que l'algorithme est déterministe — seulement que 3 tirages ont donné la même chose.
2. **Les trois configurations varient plusieurs paramètres à la fois.** On ne peut donc pas attribuer l'écart à un paramètre précis. Un vrai plan d'expérience ferait varier un paramètre à la fois (*one-factor-at-a-time*) ou utiliserait un plan factoriel.
3. **Une seule instance.** Nos conclusions valent pour ces 20 villes précises.
4. **Pas de test de significativité.** Nous ne pouvons pas affirmer que la différence entre configurations est statistiquement significative.

Ce qu'il faudrait faire : 30 seeds minimum par configuration, un plan factoriel sur les hyperparamètres, et un test de Wilcoxon ou de Mann-Whitney pour comparer les distributions.

À dire à l'oral : « Notre benchmark suffit à départager nos trois configurations de façon argumentée, mais il ne suffit pas à faire une affirmation statistique. Nous le disons explicitement dans nos limites. »

PARTIE 20 — Analyse comparative des résultats

20.1 Les chiffres

	Christofides	Algorithme génétique
Distance	3 445,60 km	3 157,13 km
Différence absolue	—	288,48 km de moins
Différence relative	—	8,37 % de moins
Temps observé	≈ 1,5 ms	≈ 1,2 s par exécution
Facteur de temps	1×	≈ 800 à 2 000× plus lent

Calculs vérifiés :

$3\,445,604687 - 3\,157,127206 = 288,477482$ km
 $288,477482 / 3\,445,604687 = 0,0837233 \rightarrow 8,37 \%$

20.2 Comparaison sur les quatre critères du sujet

Le sujet demande explicitement : **distance totale, temps d'exécution, facilité d'implémentation, robustesse de solution.**

Distance totale

Le génétique gagne, de 288,48 km (8,37 %). Sur une tournée que Théobald répète régulièrement, c'est un gain réel et mesurable.

Temps d'exécution

Christofides écrase le génétique. ≈ 1,5 ms contre ≈ 1,2 s : un facteur d'environ 1 000. Et ce n'est pas un accident : Christofides fait un nombre fixe et faible d'opérations, tandis que le génétique évalue $160 \times 520 = \mathbf{83\,200}$ tournées.

Facilité d'implémentation

C'est le critère le plus nuancé, et il faut le dire ainsi :

	Christofides	Génétique
Nombre de briques	6 (Prim, degrés, couplage, union, Hierholzer, raccourcis)	4 opérateurs + une boucle
Difficulté conceptuelle	élevée : il faut comprendre parité, Euler, couplage	modérée : la métaphore est intuitive
Piège principal	le couplage (blossom) est très difficile à écrire soi-même	la validité de la permutation après croisement
Lignes de code	~120	~150
Une fois écrit	rien à régler	6 hyperparamètres à calibrer
Dépendance externe	NetworkX pour le couplage	aucune (juste random)

Conclusion nuancée : « Christofides est plus long à écrire et demande plus de bagage théorique, mais une fois écrit il n'y a plus rien à faire. Le génétique s'écrit plus vite, mais le vrai travail commence après : il faut le régler, le benchmarker, choisir une configuration. Au total, le temps passé a été comparable. »

Robustesse

- **Christofides** : parfaitement déterministe. Même entrée → même sortie, toujours.

Robustesse maximale par construction.

- **Génétique** : dépend du seed. Notre configuration retenue a donné $\sigma = 0$ km sur 3

seeds, ce qui est excellent — mais sur 3 seeds seulement. Les deux autres configurations montrent $\sigma = 83,48$ km, c'est-à-dire un risque réel d'obtenir 3 334,21 km, **soit moins bien que Christofides**.

La phrase à retenir : « Le génétique est meilleur en moyenne et au mieux, mais il peut aussi être pire. Christofides ne peut jamais surprendre. »

20.3 Avantages et inconvénients pour Théobald

Christofides

Avantages — instantané ; déterministe ; garantie $\leq 1,5 \times \text{OPT}$; aucun réglage ; explicable de bout en bout ; passe à l'échelle sans réglage. **Inconvénients** — ici 8,37 % plus long ; ne peut pas être amélioré en lui donnant plus de temps de calcul ; nécessite l'inégalité triangulaire pour garder sa garantie ; le couplage blossom est coûteux ($O(n^3)$) sur de grosses instances.

Algorithme génétique

Avantages — meilleure distance observée ; s'améliore si on lui donne plus de temps ; s'adapte facilement à de nouvelles contraintes (fenêtres horaires, capacité, plusieurs véhicules) en changeant juste la fitness ; ne suppose rien sur la métrique. **Inconvénients** — aucune garantie ; dépend du seed ; 6 hyperparamètres à régler ; $\sim 1\,000\times$ plus lent ; peut tomber sur un optimum local médiocre (nos 3 334,21 km).

20.4 Pourquoi le génétique peut battre Christofides

C'est la question de compréhension du projet. Réponse :

« La garantie de Christofides est un **plafond**, pas un **plancher**. Elle dit "je ne ferai jamais pire que $1,5 \times \text{OPT}$ ". Elle ne dit pas "je ferai mieux que tout le monde". Christofides suit une construction imposée — un MST, un couplage, un ordre eulérien — et cette construction est rigide : elle ne peut pas revenir en arrière pour améliorer un croisement d'arêtes. Le génétique, lui, explore librement l'espace des permutations et peut tomber sur un arrangement que la construction de Christofides ne pouvait pas produire. »

Le contraste visuel à montrer sur les cartes : dans le sud-est, Christofides fait Nîmes → Nice → Toulon → Marseille → Saint-Étienne, alors que le génétique fait Nîmes → Marseille → Toulon → Nice → Grenoble. Christofides « monte » à Nice puis « redescend » vers Marseille : deux arêtes se croisent. Le génétique les a décroisées. **C'est là que se jouent les 288 km.**

20.5 Ce que ce résultat ne prouve PAS

□ « Les algorithmes génétiques sont meilleurs que Christofides. » □ « Sur cette instance de 20 villes, avec nos réglages, notre génétique a trouvé une tournée 8,37 % plus courte que celle de notre Christofides. »

Trois raisons de rester prudent :

1. **Une seule instance.** Sur un autre jeu de villes, le classement peut s'inverser.

2. **Nos seeds défavorables donnent 3 334,21 km**, c'est-à-dire **moins bien** que Christofides.
3. **Christofides a une garantie**, le génétique n'en a aucune. À 1 000 villes, sans temps de calcul supplémentaire, Christofides resterait borné, pas le génétique.

PARTIE 21 — Les bornes sur l'optimum

21.1 Ce que nous savons

2 665,05 km ≤ OPT ≤ 3 157,13 km
MST meilleure tournée valide trouvée

Fenêtre d'incertitude : **492,07 km**.

21.2 Pourquoi le MST est une borne inférieure valable

Démontré en §4.4. C'est une **preuve**, pas une estimation.

21.3 Pourquoi notre meilleure tournée est une borne supérieure

Trivialement : c'est une tournée **valide** (elle visite les 20 villes une fois et revient au départ — nos tests unitaires le vérifient). L'optimum, étant le minimum sur toutes les tournées valides, est donc $\leq$ à celle-ci.

21.4 La borne déduite de la garantie de Christofides

Christofides $\leq 1,5 \times \text{OPT}$ □ $\text{OPT} \geq 3\,445,60 / 1,5 = 2\,297,07 \text{ km}$

Cette borne est moins bonne que celle du MST ($2\,297,07 < 2\,665,05$). On garde donc la meilleure des deux : 2 665,05 km.

C'est un point d'analyse fin à sortir : « La garantie de Christofides nous donne aussi une borne inférieure, mais elle est plus faible que celle du MST. C'est logique : la garantie est faite pour le pire cas, et notre instance est loin du pire cas. »

21.5 Ce que ça permet de dire sur la qualité

Méthode	Distance	Rapport à la borne inférieure	Écart maximal à l'optimum
MST (borne)	2 665,05	1,000	—
Génétique	3 157,13	1,185	$\leq 18,5 \%$
Christofides	3 445,60	1,293	$\leq 29,3 \%$

Ces majorations sont **garanties**, sans connaître l'optimum. Et elles sont bien meilleures que la garantie théorique de 50 % de Christofides.

21.6 Pourquoi on ne connaît toujours pas l'optimum

Parce que nous n'avons pas résolu le problème **exactement**. Le faire demanderait :

- **Held-Karp** (programmation dynamique) : $O(n^2 \cdot 2^n)$. Pour $n = 20$:

$400 \times 1\,048\,576 \approx 4,2 \times 10^8$ opérations et surtout **~168 Mo** de table minimum en théorie, bien plus en Python. C'est à la limite du faisable, mais pas en Python pur.

- **Branch and bound** ou **branch and cut** : faisable, mais c'était hors du périmètre du sujet.

Réponse honnête si on vous demande pourquoi vous ne l'avez pas fait : « Le sujet demande explicitement de comparer deux méthodes approchées et de ne pas tout évaluer. Calculer l'optimum exact aurait été un projet en soi. Nous préférons encadrer l'optimum honnêtement plutôt que de prétendre l'avoir trouvé. »

PARTIE 22 — Complexité

22.1 Big-O, en une minute

La notation $O(f(n))$ décrit **comment le coût grandit** quand la taille de l'entrée grandit, en ignorant les constantes et les termes négligeables.

- $O(1)$: constant.
- $O(\log n)$: très lent à grandir (doubler n ajoute une étape).
- $O(n)$: proportionnel.
- $O(n \log n)$: un tri efficace.
- $O(n^2)$: doubler n quadruple le coût.
- $O(n^3)$: doubler n multiplie par 8.
- $O(2^n)$, $O(n!)$: **explosif**, inutilisable au-delà de petites tailles.

22.2 Notre projet, brique par brique

Étape	Complexité	Chez nous ($n = 20$)
Lecture du CSV	$O(n)$	20 lignes
Matrice des distances	$O(n^2)$	190 appels à Haversine
Construction du graphe	$O(n^2)$	190 arêtes
Prim (tas binaire)	$O(E \log V) = O(n^2 \log n)$	≈ 820 opérations de tas
Calcul des degrés	$O(n)$	19 arêtes parcourues
Construction du graphe des impairs	$O(k^2)$, $k = 12$	66 arêtes
Couplage (blossom, NetworkX)	$O(k^3)$	$\approx 1\,700$ opérations
Union	$O(n)$	25 arêtes
Hierholzer	$O(E)$	25 arêtes
Shortcutting	$O(E)$	25 sommets parcourus
Christofides total	dominé par $O(n^2 \log n)$ et $O(k^3)$	$\approx 1,5$ ms mesuré

Algorithme génétique

Étape	Complexité
Une évaluation de fitness	$O(n)$ — 20 additions
Évaluation d'une génération	$O(P \cdot n)$
Tri de la population	$O(P \log P)$
Un croisement OX	$O(n^2)$ (à cause du <code>gene not in child</code>)
Une mutation	$O(n)$
Total	$O(G \cdot P \cdot n^2)$ dans le pire cas, dominé en pratique par $O(G \cdot P \cdot n)$

Chiffres concrets :

- Évaluations de fitness : $520 \times 160 = 83\,200$
- Opérations élémentaires pour la fitness : $83\,200 \times 20 \approx 1,66$ million
- Croisements : environ $520 \times 77 \approx 40\,000$, chacun en $O(19^2) \approx 361 \rightarrow \approx 14$ millions

d'opérations

C'est ce qui explique les $\sim 1,2$ s, contre 1,5 ms pour Christofides.

22.3 Ce qui coûte vraiment du temps

Pour Christofides : le couplage. Sur 12 sommets c'est instantané, mais sur une instance de 1 000 villes avec ~ 500 sommets impairs, $O(k^3) = 1,25 \times 10^8$ devient la partie dominante.

Pour le génétique : les 83 200 évaluations de fitness, en Python pur (pas de NumPy vectorisé). C'est le poste numéro un, et c'est aussi le plus facile à optimiser.

22.4 Une nuance importante sur le GA

Le coût du génétique **ne dépend pas de la difficulté du problème**, mais uniquement des paramètres qu'on lui donne : $G \times P$. Il fera 83 200 évaluations que l'instance soit facile ou difficile. C'est une différence fondamentale avec un algorithme exact, dont le coût explose avec la difficulté.

Corollaire : on peut **choisir** son budget de calcul, ce qui est un avantage pratique réel — mais la qualité n'est jamais garantie.

PARTIE 23 — Code walkthrough, fichier par fichier

23.1 Arborescence

```
travelling-merchant/
├── data/villes_france_lat_long.csv    20 villes, lat/lon
├── src/
│   ├── core.py                      données, Haversine, matrice, graphe
│   ├── christofides.py              Prim, couplage, Hierholzer, Christofides
│   ├── genetic.py                   GA complet + benchmark
│   └── visualization.py             cartes Folium + graphiques Matplotlib
├── tests/test_algorithms.py
├── main.py                          orchestration + export JSON/CSV/PNG/HTML
├── results/                          sorties générées
└── requirements.txt
```

23.2 `src/core.py`

City — `dataclass` gelée

```
@dataclass(frozen=True)
class City:
    name: str
    latitude: float
    longitude: float
```

Entrée / sortie. Trois champs typés. **Pourquoi une `dataclass`.** Elle génère `__init__`, `__repr__` et `__eq__` automatiquement. Le code est plus lisible que des tuples (`nom`, `lat`, `lon`) : on écrit `city.latitude` et non `city[1]`. **Pourquoi `frozen=True`.** L'objet devient **immuable** et **hashable**. Une ville ne doit jamais être modifiée en cours de calcul — si un bug essayait de le faire, Python lèverait une exception au lieu de corrompre silencieusement les distances.

Question probable. « *Pourquoi pas un dictionnaire ?* » → « Une `dataclass` donne des attributs typés, l'autocomplétion, et l'immuabilité. Un dict ne protège rien. »

`load_cities`

```
def load_cities(csv_path) -> list[City]:
    path = Path(csv_path)
    if not path.exists():
        raise FileNotFoundError(f"CSV introuvable: {path}")
    with path.open("r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        expected = {"Ville", "Latitude", "Longitude"}
        if set(reader.fieldnames or []) != expected:
            raise ValueError(...)
        ...
    if len(cities) < 3:
        raise ValueError("Le TSP nécessite au moins 3 villes.")
    return cities
```

Entrée. Un chemin. **Sortie.** Une liste de `City` dans l'ordre du fichier.

Trois validations, chacune justifiable :

1. **Fichier présent**, sinon message clair.
2. **Colonnes exactement** `{Ville, Latitude, Longitude}` — pas juste « contient ». Un fichier avec une colonne en trop ou mal nommée est refusé tout de suite, au lieu de planter 50 lignes plus loin.
3. **Au moins 3 villes** : en dessous, la notion de cycle TSP n'a pas de sens.

`encoding="utf-8-sig"`. Gère le **BOM** (Byte Order Mark) que Windows/Excel ajoute en tête des CSV. Sans ça, la première colonne s'appellerait " `Ville`" et la validation échouerait. C'est un détail très concret à savoir expliquer.

`newline=""`. Recommandation officielle du module `csv` : laisse le module gérer lui-même les fins de ligne.

L'ordre du fichier fixe les indices. Paris est la première ligne, donc l'indice **0**, donc l'ancre du GA. C'est une dépendance implicite qu'il faut connaître.

`haversine_km`

Détaillée en Partie 3. Entrée : deux `City`. Sortie : un `float` en km.

`distance_matrix`

```
def distance_matrix(cities) -> list[list[float]]:
    n = len(cities)
    matrix = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            d = haversine_km(cities[i], cities[j])
            matrix[i][j] = d
            matrix[j][i] = d
    return matrix
```

Pourquoi seulement le triangle supérieur. `j` démarre à `i+1`, donc on ne calcule chaque paire **qu'une fois** — 190 appels au lieu de 400 — puis on **miroite** (`matrix[j][i] = d`). La symétrie est donc garantie par construction, pas espérée.

Pourquoi une matrice plutôt que recalculer. Le GA évalue 83 200 tournées × 20 segments = **1,66 million** de lectures de distance. Recalculer Haversine (avec ses sinus et cosinus) à chaque fois serait catastrophique. Là, c'est un simple accès mémoire. **C'est la principale optimisation du projet.**

Question probable. « *Pourquoi pas NumPy ?* » → « Une liste de listes 20×20 suffit largement et évite une dépendance sur le cœur du calcul. Sur une instance de plusieurs milliers de villes, NumPy deviendrait indispensable. »

`build_complete_graph`

```
def build_complete_graph(cities, matrix=None) -> nx.Graph:
    matrix = matrix or distance_matrix(cities)
    graph = nx.Graph()
    for i, city in enumerate(cities):
        graph.add_node(i, name=city.name, latitude=..., longitude=...)
    for i in range(len(cities)):
        for j in range(i + 1, len(cities)):
            graph.add_edge(i, j, weight=matrix[i][j])
    return graph
```

Pourquoi NetworkX ici. Parce que `prim_mst` prend un graphe, et surtout parce que le **couplage** de NetworkX en a besoin. On réutilise la matrice si elle est déjà calculée (`matrix or ...`) pour ne pas refaire les 190 Haversine.

Les attributs de nœud (`name`, `latitude`, `longitude`) ne servent pas aux algorithmes, mais ils rendent le graphe auto-descriptif si on l'inspecte ou l'exporte.

`route_distance` **et** `route_names`

```
def route_distance(route, matrix) -> float:
    route_list = list(route)
    if len(route_list) < 2:
        return 0.0
    return sum(matrix[route_list[i]][route_list[i + 1]]
                for i in range(len(route_list) - 1))

def route_names(route, cities) -> list[str]:
    return [cities[i].name for i in route]
```

`route_distance` **ne referme pas le cycle** : elle somme ce qu'on lui donne. C'est l'appelant qui doit fournir une route déjà fermée. C'est un choix assumé : la fonction reste utilisable pour un chemin ouvert.

`route_names` traduit les indices en noms — uniquement pour l'affichage et le JSON.

23.3 `src/christofides.py`

ChristofidesResult

```
@dataclass
class ChristofidesResult:
    route: list[int]
    distance_km: float
    mst_weight_km: float
    matching_weight_km: float
    odd_vertices: list[int]
```

Pourquoi renvoyer autant de choses. Parce que le sujet demande d'**expliquer les étapes**. En exposant le poids du MST, celui du couplage et la liste des sommets impairs, on peut vérifier et présenter chaque étape intermédiaire — au lieu de sortir un nombre magique. C'est directement ce qui alimente le `summary.json` et les slides.

Elle n'est **pas frozen** : c'est un simple conteneur de résultats, pas une valeur partagée qu'on risque de muter par accident.

prim_mst

Détaillée en Partie 5.

_hierholzer_multigraph

Détaillée en Partie 7. Le `_` initial signale une fonction **privée** au module — elle n'est pas destinée à être appelée de l'extérieur. Même convention pour `_tournament`, `_ordered_crossover`, `_mutate`, `_closed_distance`.

christofides

Détaillée en Partie 6.

23.4 `src/genetic.py`

GAConfig

```
@dataclass(frozen=True)
class GAConfig:
    population_size: int = 300
    generations: int = 1200
    tournament_size: int = 5
    crossover_rate: float = 0.95
    mutation_rate: float = 0.22
    elite_size: int = 8
    seed: int = 42
```

Pourquoi une dataclass gelée. Tous les réglages sont regroupés en un seul objet, ce qui rend les signatures propres (`genetic_tsp(cities, config)`) et rend impossible la modification accidentelle d'un paramètre au milieu d'une exécution — ce qui ruinerait la reproductibilité.

Piège à connaître. Les valeurs par défaut (300 / 1200) **ne sont pas** celles utilisées dans le projet. `main.py` passe explicitement ses trois configurations, et celle qui est retenue est `population_size=160, generations=520`. Si le professeur ouvre `genetic.py` et lit 300/1200, sachez répondre : « ce sont les valeurs par défaut de la dataclass ; les configurations réellement utilisées sont définies dans `main.py`, et le `summary.json` donne celle qui a été retenue. »

GAResult

Contient `route`, `distance_km`, `history` et `config`. Renvoyer `history` est ce qui permet de tracer la courbe de convergence ; renvoyer `config` permet de savoir exactement avec quels réglages le résultat a été obtenu.

Les opérateurs

`_closed_distance`, `_tournament`, `_ordered_crossover`, `_mutate` — détaillés dans les parties 12 à 15.

`genetic_tsp`

Détaillée en Partie 10. Notez les deux garde-fous en tête de fonction :

```
if config.population_size < 4:
    raise ValueError("population_size doit être >= 4")
if config.elite_size >= config.population_size:
    raise ValueError("elite_size doit être inférieur à population_size")
```

Le second est indispensable : si l'élite remplissait toute la population, la boucle `while len(new_population) < population_size` ne s'exécuterait jamais et l'algorithme resterait figé.

`benchmark_configs`

Détaillée en Partie 19.

23.5 `src/visualization.py`

`save_route_map` — **carte interactive Folium**

```
m = folium.Map(location=center, zoom_start=6, tiles="CartoDB positron")
for order, idx in enumerate(route[:-1], start=1):
    folium.CircleMarker(..., radius=6, tooltip=f"{order}. {city.name}", ...).add_to(m)
points = [[cities[idx].latitude, cities[idx].longitude] for idx in route]
folium.PolyLine(points, weight=4, opacity=0.8, tooltip=title).add_to(m)
```

Pourquoi Folium. Il produit un fichier **HTML autonome** avec une vraie carte Leaflet, ouvrable dans n'importe quel navigateur, sans installer quoi que ce soit. C'est la meilleure réponse possible à « affichez l'itinéraire sur la carte du marchand ».

`route[:-1]` évite de poser deux fois le marqueur sur Paris (présent au début **et** à la fin). En revanche la `PolyLine` utilise `route` **entier**, pour que le segment de retour soit bien tracé.

`center` est le barycentre des 20 villes, pour cadrer automatiquement.

`save_route_png` **et** `save_ga_history` — **Matplotlib**

Pourquoi Matplotlib. Pour obtenir des **images statiques** insérables dans un rapport ou des slides, et pour la courbe de convergence qui n'est pas géographique.

Pourquoi les deux. HTML = exploration interactive ; PNG = document imprimable. Les deux sont demandés implicitement par « présentation synthétique des résultats ».

*Pour la soutenance, nous avons **regénéré** ces visuels avec une charte graphique dédiée (`tools/`), en réutilisant **exactement les mêmes données**. Les figures du PowerPoint sont donc cohérentes au chiffre près avec `results/`.*

23.6 `main.py`

```
def run_all() -> dict:
    cities = load_cities(DATA)

    start = time.perf_counter()
    christ = christofides(cities)
    christ_time = time.perf_counter() - start

    configs = {
        "rapide": GAConfig(population_size=80, generations=220, tournament_size=4, mutation_rate=0.18, eli
        "equilibree": GAConfig(population_size=160, generations=520, tournament_size=5, mutation_rate=0.22, eli
        "exploratoire": GAConfig(population_size=240, generations=800, tournament_size=6, mutation_rate=0.30, eli
    }

    benchmark = benchmark_configs(cities, configs, seeds=[11, 22, 33])
    best_config_name = min(benchmark, key=lambda row: (row["mean_km"], row["std_km"], row["best_km"]))['configura
    chosen = configs[str(best_config_name)]

    ga_runs = []
    for seed in [11, 22, 33]:
        config = GAConfig(**asdict(chosen), "seed": seed)
        result = genetic_tsp(cities, config)
        ga_runs.append((result, elapsed))
    ga, ga_best_time = min(ga_runs, key=lambda item: item[0].distance_km)

    # ... cartes, PNG, CSV, JSON
```

Ce que `main.py` orchestre :

1. Charge les données.
2. Exécute Christofides, **chronométré**.
3. Définit **trois** configurations génétiques.
4. Lance le benchmark ($3 \times 3 = 9$ exécutions).
5. **Choisit** la meilleure configuration par un critère explicite.
6. Relance cette configuration sur 3 seeds et garde le meilleur résultat.
7. Génère 2 cartes HTML, 3 PNG, 1 CSV et 1 JSON.

`time.perf_counter()` et non `time.time()` : c'est l'horloge monotone de plus haute résolution, conçue pour mesurer des **durées**. `time.time()` peut sauter si l'horloge système est ajustée.

`GAConfig(**asdict(chosen), "seed": seed)` : on décompose la config choisie en dictionnaire, on écrase **uniquement** le seed, et on reconstruit une config gelée. C'est l'équivalent propre d'un `dataclasses.replace`.

`--json` affiche le résumé complet sur la sortie standard, pratique pour une démo.

23.7 tests/test_algorithms.py

```
def _assert_valid_cycle(self, route):
    self.assertEqual(route[0], route[-1]) # (1)
    self.assertEqual(len(route), len(self.cities) + 1) # (2)
    self.assertEqual(set(route[:-1]), set(range(len(self.cities)))) # (3)
    self.assertEqual(len(set(route[:-1])), len(self.cities)) # (4)
```

Ce que les tests garantissent :

1. La tournée **revient au point de départ**.
2. Elle a la **bonne longueur** : 20 villes + le retour = 21 éléments.
3. Elle contient **exactement** les 20 villes, ni plus ni moins.
4. **Aucun doublon**.
5. La distance est strictement positive.

Et ceci **pour les deux algorithmes**.

Ce que les tests NE garantissent PAS — à assumer clairement :

- Ils ne vérifient **pas** que la distance est optimale, ni même bonne.
- Ils ne vérifient **pas** la valeur du MST ni celle du couplage.
- Ils ne vérifient **pas** la formule de Haversine (pas de valeur de référence).
- Ils ne vérifient **pas** la non-régression : aucune valeur attendue n'est figée.
- Le GA est testé avec une configuration réduite (60 individus, 80 générations) pour

aller vite, donc pas dans les conditions réelles.

Question probable : « *Comment savez-vous que votre code est correct ?* » → « Trois niveaux. Un : les tests unitaires garantissent la **validité structurelle** des tournées. Deux : nous avons fait des vérifications croisées — le poids de notre MST Prim coïncide exactement avec `networkx.minimum_spanning_tree`, et il est identique quel que soit le sommet de départ. Trois : nous avons vérifié que la matrice est symétrique, de diagonale nulle, et qu'aucun des 6 840 triplets ne viole l'inégalité triangulaire. Ce qu'il manque, ce sont des tests de non-régression sur les valeurs, et un test de Haversine contre une distance de référence connue. C'est la première chose que nous ajouterions. »

PARTIE 24 — Pourquoi avons-nous fait ces choix ?

Choix	Justification défendable
Graphe complet	On sait calculer Haversine entre toute paire de villes : toutes les liaisons existent. Et c'est ce qui garantit qu'un cycle hamiltonien existe.
Pondéré	Sans poids, « le plus court chemin » n'a pas de sens.
Non orienté	Haversine est symétrique : $d(A,B) = d(B,A)$. Modéliser un graphe orienté doublerait les arêtes sans rien apporter.
Haversine	Imposé par le sujet, et correct : nos données sont des coordonnées sphériques. C'est une métrique, ce dont Christofides a besoin.
Christofides	Imposé par le sujet. Pertinent car nos distances sont métriques (vérifié) et qu'il apporte une garantie.
Prim	Cité par la base de connaissances du sujet ; adapté aux graphes denses ; ne nécessite qu'un tas, pas d'union-find.
Un MST	Première étape de Christofides, et fournit en prime la borne inférieure $MST \leq OPT$.
Un couplage	Seul moyen de rendre pairs tous les degrés impairs en ajoutant le minimum de poids.
Couplage <i>minimal</i>	(a) coûte moins cher ; (b) c'est la minimalité qui permet de prouver $W \leq OPT/2$.
Hierholzer	Algorithme standard du circuit eulérien, en $O(E)$, simple à écrire en itératif.
Un algorithme génétique	Imposé par le sujet ; c'est aussi la métaheuristique la plus naturelle quand une solution est une permutation.
Tournoi	Ne compare que des ordres, donc aucun besoin de transformer la fitness ; un seul paramètre lisible ; robuste à l'échelle des valeurs.
OX	Garantit une permutation valide sans réparation , et transmet l'ordre relatif, qui est l'information pertinente pour une tournée.

Choix	Justification défendable
Mutation swap + inversion	L'inversion est un mouvement 2-opt, très efficace sur le TSP ; le swap ajoute une perturbation différente. Les deux préservent la permutation.
Élitisme	Garantit qu'on ne perd jamais la meilleure solution trouvée. Limité à 3,75 % pour ne pas tuer la diversité.
Paris fixé	Supprime les 20 écritures redondantes du même cycle sans perdre aucune solution.
Plusieurs seeds	Une exécution unique d'un algorithme stochastique ne dit rien sur sa fiabilité.
Trois configurations	Demandé explicitement par le sujet : « testez différentes configurations ».
Benchmark	Transforme un choix d'intuition en choix mesuré et reproductible.
NetworkX seulement par endroits	Pour le couplage (blossom, $O(n^3)$, très difficile à écrire correctement) et comme structure de graphe. Pas pour Prim, Hierholzer ni Christofides — le sujet demande de les implémenter et de les expliquer.
Matrice de distances précalculée	1,66 million de lectures de distance dans le GA : recalculer Haversine à chaque fois serait rédhibitoire.
Folium + Matplotlib	HTML interactif pour explorer, PNG statique pour les documents.
JSON + CSV en sortie	Résultats exploitables par un humain et par un programme, sans relancer le calcul.
<code>unittest</code>	Bibliothèque standard, aucune dépendance supplémentaire ; suffisant pour valider la structure des tournées.

24.1 La question « pourquoi NetworkX seulement à certains endroits ? »

C'est une question que le professeur posera probablement. Réponse structurée :

« Nous avons implémenté nous-mêmes tout ce que le sujet demande de comprendre : Prim, la détection des degrés impairs, l'union en multigraphe, Hierholzer et le shortcutting. Nous avons délégué à NetworkX uniquement le couplage parfait de poids minimal, parce que c'est l'algorithme des fleurs d'Edmonds — une brique d'optimisation combinatoire à part entière, en $O(n^3)$, dont la réimplémentation n'aurait rien appris sur le TSP. NetworkX nous sert aussi de structure de graphe. Le sujet cite d'ailleurs NetworkX dans sa base de connaissances. »

Bonus. NetworkX possède `networkx.approximation.christofides`. Nous ne l'avons **pas** utilisé — nous avons écrit le nôtre. Dites-le : c'est un point fort.

PARTIE 25 — Les alternatives, et pourquoi pas elles

Méthode	Ce que c'est	Pourquoi pas ici
Force brute	énumérer les $(n-1)!$ tournées	$1,2 \times 10^{17}$ tournées $\approx$ 3,9 ans à 10^9 /s. Le sujet l'interdit explicitement.

Méthode	Ce que c'est	Pourquoi pas ici
Plus proche voisin	aller toujours à la ville non visitée la plus proche	Très rapide, mais typiquement 25 % au-dessus de l'optimum , sans garantie utile (la borne théorique est en $O(\log n) \times OPT$). Bon comme point de comparaison, pas comme réponse.
Insertion (nearest / cheapest)	insérer les villes une à une au meilleur endroit	Garantie de facteur 2, donc moins bonne que Christofides.
2-opt	décroiser deux arêtes, répéter jusqu'à stabilité	Excellent, et complémentaire : c'est une recherche locale , pas une méthode de construction. Ce serait notre amélioration n° 1.
3-opt / Or-opt	idem avec 3 arêtes ou déplacement de segments	Meilleur que 2-opt, plus coûteux. Même statut.
Lin-Kernighan	recherche locale à profondeur variable	L'état de l'art heuristique. Hors périmètre d'un projet de ce format.
Held-Karp	programmation dynamique exacte	$O(n^2 \cdot 2^n)$. Pour $n = 20$: $\sim 4 \times 10^8$ opérations et une table de plusieurs centaines de Mo. Infaisable en Python pur ; faisable à $n \leq 15$.
Branch and bound	exploration avec élagage	Exact et praticable à 20 villes, mais c'est un projet entier, et le sujet demande deux méthodes précises.
Recuit simulé	une seule solution, perturbée, acceptant parfois le pire	Métaheuristique valable, souvent aussi bonne que le GA sur le TSP. Le sujet impose un algorithme génétique .
Colonies de fourmis (ACO)	phéromones sur les arêtes	Très adaptée au TSP. Même remarque : le sujet impose le GA.
Dijkstra	plus court chemin d'un point à un autre	Résout un autre problème . Ne visite pas toutes les villes.
Kruskal	MST par tri des arêtes	Aurait donné le même poids de MST . Prim est simplement mieux adapté à un graphe dense et cité par le sujet.
A\	plus court chemin avec heuristique	Comme Dijkstra : autre problème.
Google Maps / OSRM	distances routières réelles	Plus réaliste, mais : dépendance réseau, quotas, non-reproductible, et surtout perte de la garantie de Christofides car l'inégalité triangulaire n'est plus assurée.
Solveur MILP (OR-Tools, Gurobi)	formulation en programmation linéaire en nombres entiers	Résoudrait l'instance à l'optimum en quelques secondes. Mais le sujet demande d' implémenter les algorithmes, pas d'appeler un solveur.

PARTIE 26 — Limites et améliorations

26.1 Les limites, honnêtement

- Distances géodésiques $\neq$ distances routières.** Nos 3 157 km sont à vol d'oiseau. Le trajet réel de Théobald serait 15 à 30 % plus long.
- Aucun optimum exact calculé.** Nous encadrons $OPT \in [2\ 665,05 ; 3\ 157,13]$ et c'est tout ce que nous pouvons affirmer.

3. **Le GA n'a aucune garantie.** Nos seeds défavorables donnent 3 334,21 km, c'est-à-dire **moins bien que Christofides**.
4. **Benchmark à 3 seeds.** Insuffisant pour une conclusion statistique.
5. **Hyperparamètres empiriques.** Aucune théorie ne justifie 160/520/5/95 %/22 %/6.
6. **Plan d'expérience imparfait.** Nos trois configurations font varier plusieurs paramètres à la fois : on ne peut pas isoler l'effet de chacun.
7. **Une seule instance, 20 villes.** Les conclusions ne se généralisent pas.
8. **Temps dépendants du matériel.** Nous avons observé un facteur 4 entre deux machines sur le GA.
9. **Symétrie de sens non éliminée** dans la représentation du GA.
10. **Tests unitaires structurels seulement.** Pas de test de non-régression sur les valeurs, pas de test de Haversine contre une référence.
11. **Pas de contraintes réelles.** Pas de fenêtres horaires, pas de capacité de chargement, pas de durée maximale journalière — alors que la vie de Théobald en est pleine.

26.2 Les améliorations, par rapport qualité/effort

Priorité	Amélioration	Gain attendu	Effort
1	2-opt après le GA	Amélioration quasi systématique, souvent 2 à 5 %	Faible : ~30 lignes
2	Christofides comme graine du GA	On part sous 3 445,60 km garanti	Très faible : injecter une tournée dans la population initiale
3	30 seeds au lieu de 3	Permet enfin de parler de moyenne et de robustesse	Faible : changer une liste, ~2 min de calcul
4	Test de Haversine contre une référence	Sécurise la brique la plus fondamentale	Très faible
5	Plan factoriel sur les hyperparamètres	Isole l'effet de chaque paramètre	Moyen
6	Held-Karp sur une sous-instance ($n \leq 15$)	Donne le vrai écart à l'optimum	Moyen
7	Distances routières réelles (OSRM en local)	Réalisme pour Théobald	Moyen : cache local obligatoire
8	Mutation adaptative	Fort taux au début, faible à la fin	Faible
9	Critère d'arrêt anticipé	Le GA converge dès la génération 67 : 450 générations sont gaspillées	Faible
10	NumPy pour la fitness	x10 sur le temps du GA	Moyen

Le détail qui frappe

Notre GA atteint son meilleur résultat à la génération 67 sur 520. Les 453 générations suivantes ne servent à rien. Un critère d'arrêt du type « stopper après 100 générations sans amélioration » diviserait le temps de calcul par 3 sans rien perdre. C'est une observation tirée directement de nos données, et c'est le genre de remarque qui montre qu'on a **regardé** ses résultats.

26.3 La combinaison Christofides + GA

C'est l'amélioration la plus élégante, et le professeur peut la suggérer :

« On peut hybrider les deux. On calcule la tournée Christofides — 3 445,60 km, garantie $\leq 1,5 \times \text{OPT}$ — et on l'injecte comme individu de la population initiale du génétique. On obtient alors le meilleur des deux mondes : la garantie de Christofides devient une borne de départ que le GA ne peut qu'améliorer, puisque l'élitisme empêche toute régression. C'est ce qu'on appelle un algorithme mémétique. »

PARTIES 27 & 28 — Les questions du professeur

Ces deux parties sont si volumineuses qu'elles vivent dans un fichier dédié :

- `QUESTIONS_PROF_100_PLUS.md` — 172 questions classées en 5 niveaux de

difficulté, chacune avec une réponse courte, une réponse développée et l'erreur à ne pas commettre. La partie 28 (questions sur le code, quand le professeur ouvre GitHub) y constitue le niveau 3 bis.

Utilisez-le en interrogation croisée : l'un lit la question, l'autre répond sans regarder.

PARTIE 29 — DANGER À L'ORAL : ce qu'il ne faut jamais dire

Cette section est la plus rentable du guide. Relisez-la juste avant de passer.

❑ NE DITES PAS	❑ DITES
« 3 157 km est l'optimum. »	« 3 157,13 km est la meilleure tournée que nous ayons observée. »
« Christofides donne toujours 1,5 fois l'optimum. »	« Christofides garantit au plus 1,5 fois l'optimum, pour un TSP métrique . »
« Christofides est un algorithme exact. »	« C'est un algorithme d' approximation avec une garantie. »
« Le MST est une tournée de 2 665 km. »	« Le MST est une structure de connexion, pas un circuit. Son poids est une borne inférieure de l'optimum. »
« Les algorithmes génétiques sont meilleurs que Christofides. »	« Sur cette instance et avec nos réglages , notre GA a trouvé 8,37 % de mieux. »
« Notre GA a convergé, donc il a trouvé l'optimum. »	« Il a convergé, c'est-à-dire qu'il ne progresse plus. Cela ne prouve pas qu'il ait atteint l'optimum global. »
« 22 % est le taux de mutation optimal. »	« C'est la valeur de notre configuration la plus stable sur nos 3 seeds. Choisie empiriquement. »
« L'algorithme met 0,68 seconde. »	« Nous avons mesuré environ une seconde sur notre machine ; c'est une mesure expérimentale, elle dépend du matériel. »
« Euler et Hamilton, c'est pareil. »	« Euler compte les arêtes , Hamilton compte les sommets . »

❏ NE DITES PAS	❏ DITES
« On utilise Dijkstra pour le MST. »	« Prim pour le MST. Dijkstra résout les plus courts chemins depuis une source, ce n'est pas le même problème. »
« Haversine donne la distance réelle. »	« Haversine donne la distance à vol d'oiseau . La route réelle est 15 à 30 % plus longue. »
« Il y a 20! tournées. »	« $(n-1)! = 19!$ une fois le départ fixé ; et $19!/2$ si on tient compte du sens. »
« Le TSP est impossible à résoudre. »	« Il est NP-difficile : aucun algorithme exact en temps polynomial n'est connu. Des solveurs exacts existent quand même pour des instances importantes. »
« On a prouvé qu'il n'existe pas d'algorithme rapide. »	« $P = NP$ est un problème ouvert . Rien n'est prouvé. »
« Notre benchmark prouve que la config équilibrée est la meilleure. »	« Sur nos 3 seeds, c'est la seule qui soit stable. 3 seeds ne suffisent pas pour une preuve statistique. »
« On a implémenté Christofides avec NetworkX. »	« Nous avons implémenté Prim, les degrés, l'union, Hierholzer et les raccourcis nous-mêmes. NetworkX n'intervient que pour le couplage. »
« Le graphe a 380 arêtes. »	« 190 : le graphe est non orienté, donc on compte des paires, pas des couples ordonnés. »
« L'élitisme, c'est comme dans la nature. »	« C'est un biais délibéré : on garantit de ne jamais perdre la meilleure solution. En biologie personne n'est immortel — mais nous optimisons, nous ne simulons pas. »
« On a fixé Paris pour que Théobald parte de Paris. »	« On a fixé Paris pour supprimer les représentations redondantes du même cycle. Le point de départ ne change pas la longueur. »
« L'écart-type de 0 prouve que l'algo est déterministe. »	« Il montre que nos 3 seeds ont convergé au même endroit. L'algorithme reste stochastique. »

29.1 Les trois réflexes de langage

1. « **observé** » / « **mesuré** » **plutôt que** « **est** ». « La distance observée est de... » plutôt que « la distance optimale est de... ».
2. « **au plus** » **plutôt que** « **égal à** ». « au plus 1,5 fois l'optimum », jamais « 1,5 fois l'optimum ».
3. « **sur cette instance** » **plutôt que** « **en général** ». Chaque fois que vous comparez les deux algorithmes, ancrez la phrase dans notre instance.

29.2 Que faire si on ne sait pas

Ne bluffez jamais. La formule qui sauve :

« Je ne suis pas certain, mais voici comment je raisonnerais : [raisonnement]. Ce que je peux affirmer avec certitude, c'est que [fait vérifié]. »

Un professeur valorise infiniment plus un raisonnement honnête qu'une affirmation fausse assénée avec assurance.

PARTIE 30 — Préparer une question inattendue

Le but n'est pas d'apprendre par cœur, mais de savoir **reconstruire** une réponse. Pour chaque concept, sachez dérouler ces six points :

1. **Définition** — ce que c'est.
2. **Intuition** — l'image mentale.
3. **Rôle** — à quoi ça sert dans notre projet.
4. **Pourquoi** — pourquoi nous l'avons choisi.
5. **Alternative** — qu'aurions-nous pu faire d'autre.
6. **Limite** — ce que ça ne fait pas.

Exemple appliqué : le MST

1. **Définition** — l'arbre couvrant de poids total minimal.
2. **Intuition** — relier toutes les villes avec le moins de câble possible, sans jamais faire de boucle.
3. **Rôle** — première étape de Christofides.
4. **Pourquoi** — c'est la structure connexe la moins chère, et son poids majore par le bas l'optimum du TSP.
5. **Alternative** — Kruskal donnerait le même poids ; Borůvka aussi.
6. **Limite** — ce n'est pas une tournée : des sommets ont un degré 1 ou ≥ 3 , et il n'y a pas de cycle.

Exemple appliqué : la mutation

1. **Définition** — une perturbation aléatoire d'un individu.
2. **Intuition** — secouer légèrement une solution pour voir si on tombe mieux.
3. **Rôle** — maintenir la diversité et échapper aux optima locaux.
4. **Pourquoi** — sans elle, le croisement ne fait que recombinaison l'existant et la population se fige.
5. **Alternative** — d'autres opérateurs : insertion, scramble, déplacement.
6. **Limite** — trop de mutation détruit l'information acquise et transforme la recherche en hasard.

Exemple appliqué : Haversine

1. **Définition** — la distance de grand cercle entre deux points d'une sphère.
2. **Intuition** — la longueur du trajet d'un avion qui volerait en ligne droite.
3. **Rôle** — fournit les poids des 190 arêtes.
4. **Pourquoi** — imposé par le sujet, correct pour des coordonnées géographiques, et c'est une métrique, donc Christofides s'applique.
5. **Alternative** — Vincenty (plus précis), projection plane, API routière.
6. **Limite** — ignore complètement le relief et le réseau routier.

Les cinq faits à pouvoir sortir en toute circonstance

Si vous ne deviez retenir que cinq choses :

1. **190 arêtes** = $20 \times 19 / 2$, parce que le graphe est non orienté.

2. **2 665,05 ≤ OPT ≤ 3 157,13** — et savoir justifier les deux bornes.
 3. **Christofides ≤ 1,5 × OPT** parce que **MST ≤ OPT** et **couplage ≤ OPT/2**, et que le raccourci est gratuit grâce à l'inégalité triangulaire.
 4. **Le nombre de sommets impairs est toujours pair**, par le lemme des poignées de main.
 5. **OX préserve la permutation** par construction : segment + ordre relatif, sans doublon possible.
-

Récapitulatif final : le projet en 10 phrases

1. Théobald doit visiter 20 villes françaises une fois chacune et rentrer : c'est un TSP.
 2. Avec $19! \approx 1,2 \times 10^{17}$ tournées possibles, la force brute est hors de portée.
 3. On modélise en graphe complet pondéré non orienté : 20 sommets, 190 arêtes, poids = distance de Haversine.
 4. Haversine est une métrique, et nous avons vérifié que l'inégalité triangulaire est respectée sur les 6 840 triplets — c'est ce qui autorise Christofides.
 5. Christofides construit un MST (2 665,05 km), apparie ses 12 sommets impairs (1 142,03 km), en tire un circuit eulérien, puis raccourcit : **3 445,60 km**, en ~1,5 ms, avec la garantie $\leq 1,5 \times \text{OPT}$.
 6. L'algorithme génétique fait évoluer 160 tournées sur 520 générations avec tournoi, Ordered Crossover, mutation swap/inversion et élitisme.
 7. Nous avons testé 3 configurations × 3 seeds ; la configuration équilibrée est la seule stable ($\sigma = 0$ km), elle a donc été retenue par le programme lui-même.
 8. Le génétique obtient **3 157,13 km**, soit **288,48 km (8,37 %) de moins** que Christofides — mais en ~1 000 fois plus de temps, et sans aucune garantie.
 9. Nous encadrons l'optimum : **2 665,05 ≤ OPT ≤ 3 157,13**, sans jamais prétendre le connaître.
 10. Recommandation : **le génétique pour tracer la route de Théobald, Christofides comme référence et filet de sécurité** — rapide, déterministe, théoriquement borné.
-
-

ANNEXE VISUELLE — les figures du projet

Toutes ces figures sont générées par `python tools/build_all_assets.py` à partir des données réelles du projet. Elles sont disponibles en pleine résolution dans `FINAL_SOUTENANCE/assets/`.

A.1 Les données et le modèle

20 villes françaises

Sommets du graphe · coordonnées GPS du CSV

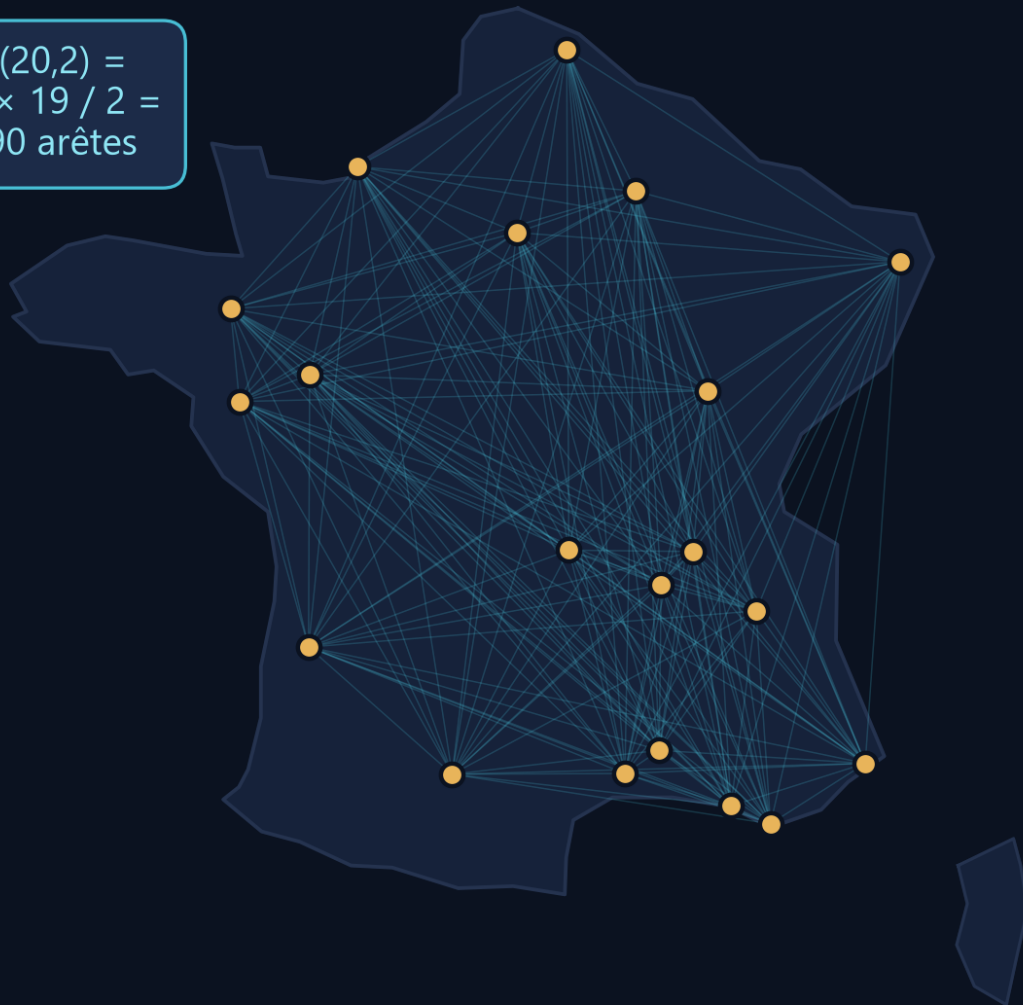


Les 20 villes du CSV, sommets du graphe.

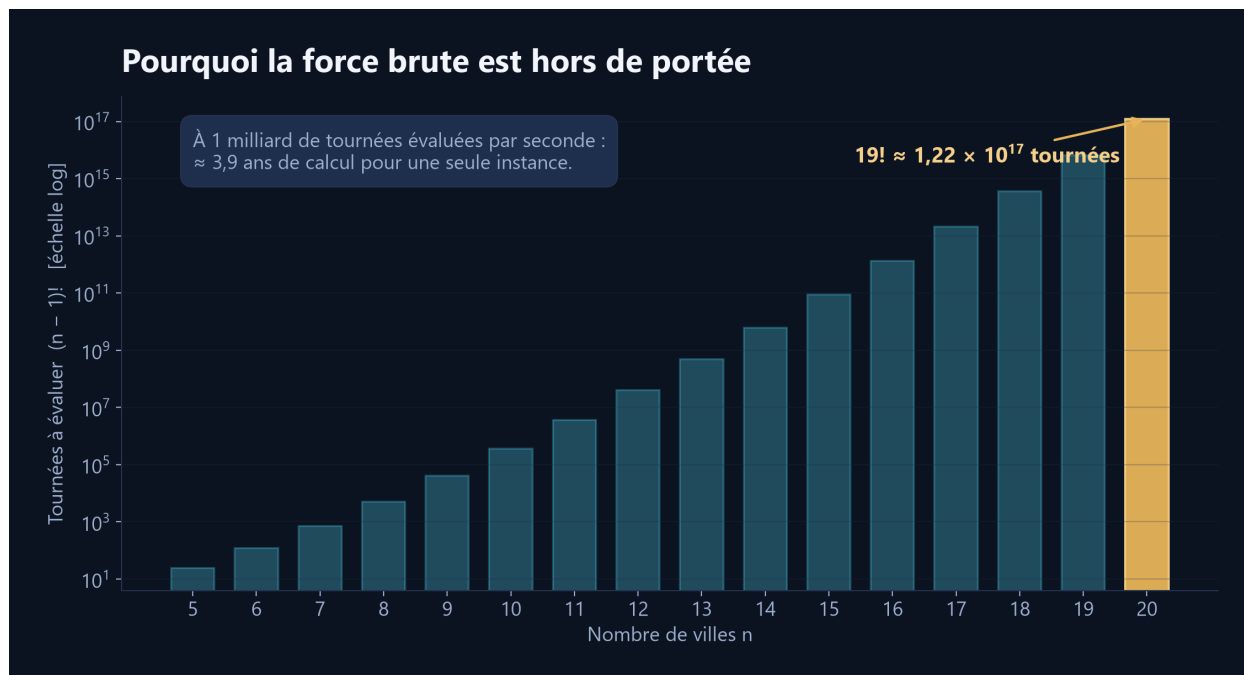
Graphe complet pondéré

20 sommets · 190 arêtes · poids = distance de Haversine

$$C(20,2) = 20 \times 19 / 2 = 190 \text{ arêtes}$$



Le graphe complet : 20 sommets, 190 arêtes, une par paire de villes.



Pourquoi la force brute est hors de portée — attention, échelle logarithmique.

Haversine : la distance sur une sphère

Les villes sont repérées en latitude / longitude, pas dans un plan.



$$d = 2R \cdot \arcsin(\sqrt{h})$$

$$h = \sin^2(\Delta\varphi/2) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2(\Delta\lambda/2)$$

$R = 6371,0088 \text{ km}$ · φ = latitude, λ = longitude (en radians)

Les 4 propriétés d'une métrique

- Non-négativité: $d(A,B) \geq 0$
- Identité: $d(A,B) = 0$ ssi $A = B$
- Symétrie: $d(A,B) = d(B,A)$
- Inégalité triangulaire: $d(A,C) \leq d(A,B) + d(B,C)$**

L'inégalité triangulaire est l'hypothèse exacte dont Christofides a besoin : c'est elle qui rend le « raccourci » gratuit.

La distance de Haversine et les quatre propriétés d'une métrique.

A.2 Christofides

Christofides : la chaîne complète

Six transformations successives — chacune est nécessaire.



Garantie : pour un TSP métrique, la tournée obtenue est au plus $1,5 \times$ l'optimum.

La chaîne complète, avec les valeurs réelles de notre instance.

Arbre couvrant minimal (Prim)

19 arêtes · poids total 2 665.05 km

n sommets
n - 1 arêtes
aucun cycle



L'arbre couvrant minimal construit par Prim — 2 665,05 km, 19 arêtes.

Sommets impairs + couplage minimal

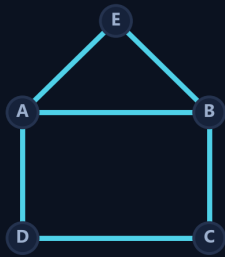
12 sommets de degré impair · couplage 1142.03 km



Les 12 sommets de degré impair et les 6 arêtes du couplage parfait de poids minimal.

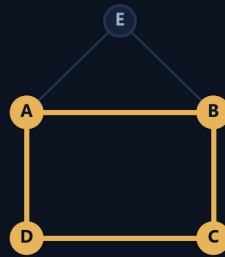
Euler, Hamilton, et le raccourci

Les trois idées qui transforment un multigraphe en tournée.



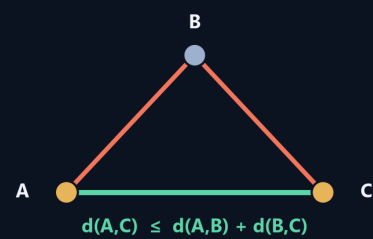
1 · Circuit EULÉRIEN

chaque ARÊTE une seule fois
c'est Hierholzer qui le construit



2 · Cycle HAMILTONIEN

chaque SOMMET une seule fois
c'est ce que cherche le TSP



3 · Le RACCOURCI

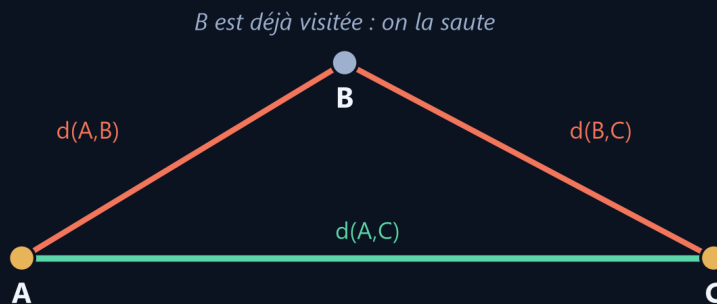
B est déjà visitée : on la saute,
sans jamais rallonger le trajet.

Aucune de ces étapes n'allonge la tournée → Christofides $\leq 1,5 \times \text{OPT}$ pour un TSP métrique

Euler compte les arêtes, Hamilton compte les sommets — et le raccourci est gratuit.

Le raccourci ne coûte jamais plus cher

C'est l'inégalité triangulaire qui rend l'étape 7 gratuite.



$$d(A,C) \leq d(A,B) + d(B,C)$$

Conséquence : convertir le circuit eulérien en cycle hamiltonien ne peut pas allonger la tournée.

L'inégalité triangulaire en détail : $d(A,C) \leq d(A,B) + d(B,C)$.

Itinéraire Christofides

Départ et retour à Paris · 20 villes visitées une seule fois

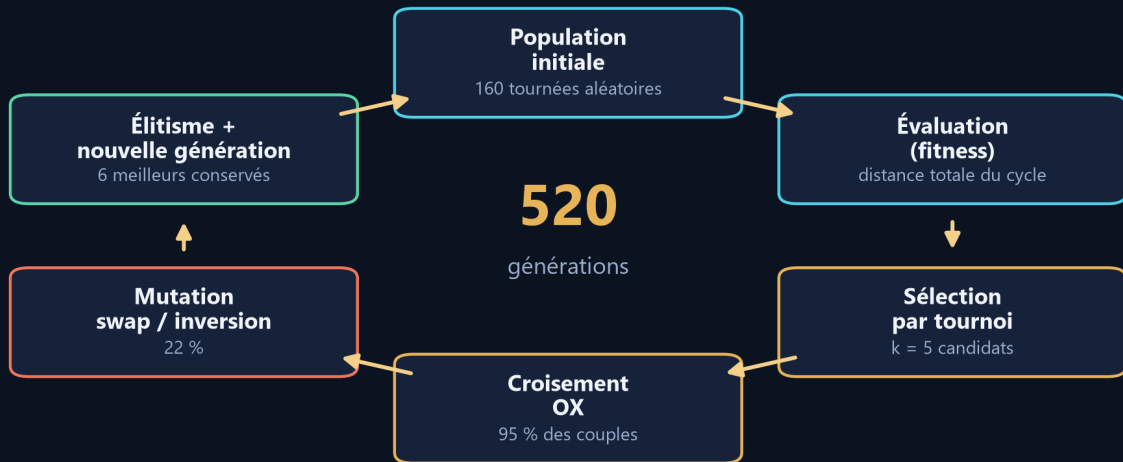


L'itinéraire final de Christofides — 3 445,60 km.

A.3 L'algorithme génétique

L'algorithme génétique : une boucle d'évolution

Une génération = une itération complète du cycle.

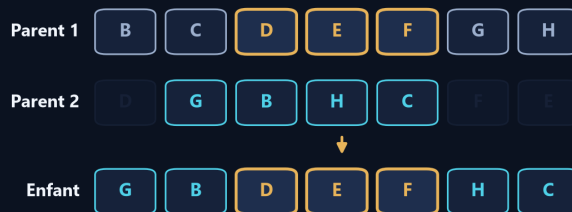


La boucle d'évolution : une génération = un tour complet.

Les quatre opérateurs de notre algorithme génétique

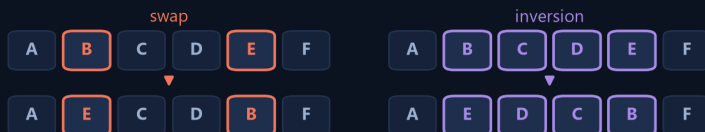
Un chromosome est une permutation : chaque opérateur doit la préserver.

1 · CROISEMENT — Ordered Crossover (OX)



Le segment du parent 1 est conservé ; les villes manquantes sont reprises dans l'ordre du parent 2. Le résultat reste une permutation valide.

2 · MUTATION — échange (swap) ou inversion de segment



3 · SÉLECTION — tournoi ($k = 5$)

Tournée 1	3412 km
Tournée 2	3289 km
Tournée 3	3877 km
Tournée 4	3157 km
Tournée 5	3602 km

parent

5 individus tirés au hasard, le meilleur devient parent. k règle la pression de sélection.

4 · ÉLITISME — 6 individus conservés

Les 6 meilleures tournées passent telles quelles à la génération suivante : la meilleure solution connue ne peut jamais être perdue en cours de route.

Les quatre opérateurs, et pourquoi chacun préserve la permutation.

Ordered Crossover (OX)

Un chromosome est une permutation : aucune ville ne doit disparaître ni apparaître deux fois.

Parent 1



on garde le segment

Parent 2



on lit l'ordre des villes restantes

Enfant



permutation valide

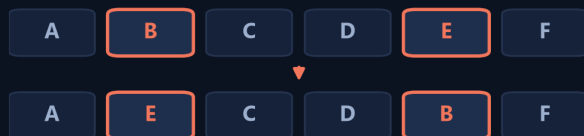
Un croisement « naïf » (couper/coller deux moitiés) créerait des doublons et des villes manquantes : la tournée serait invalide.

Le croisement OX en détail.

Mutation et sélection

Explorer sans casser la permutation, choisir sans écraser la diversité.

Mutation par échange (swap)



Mutation par inversion de segment



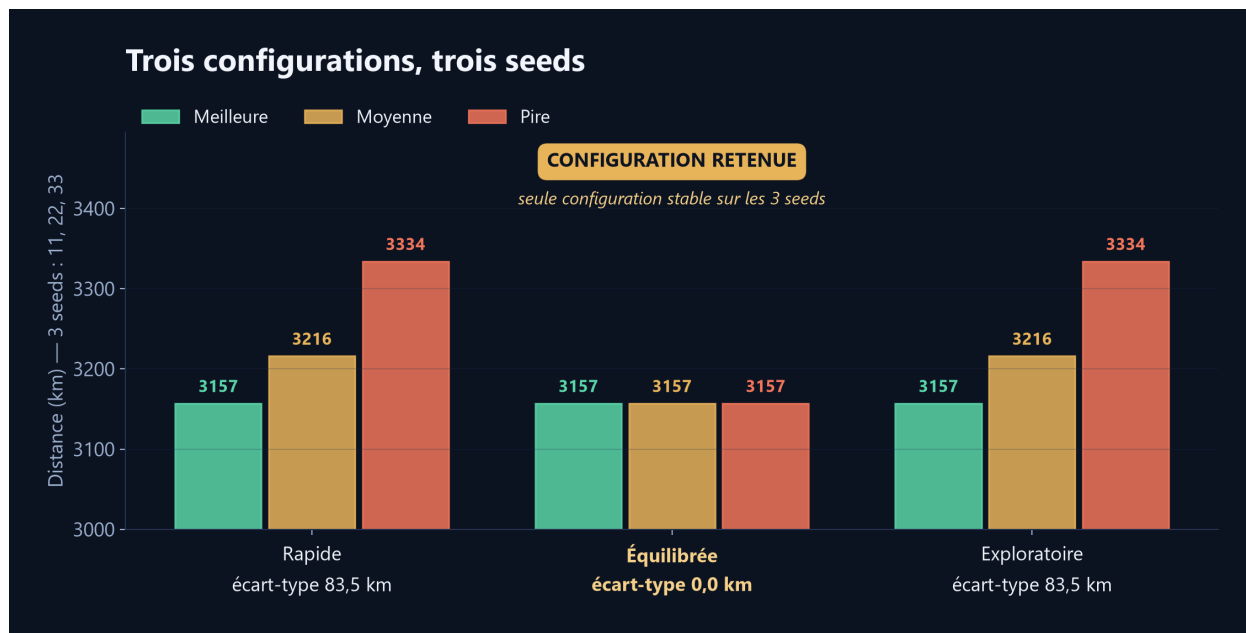
Sélection par tournoi (k = 5)

Tournée 1	3412 km
Tournée 2	3289 km
Tournée 3	3877 km
Tournée 4	3157 km
Tournée 5	3602 km

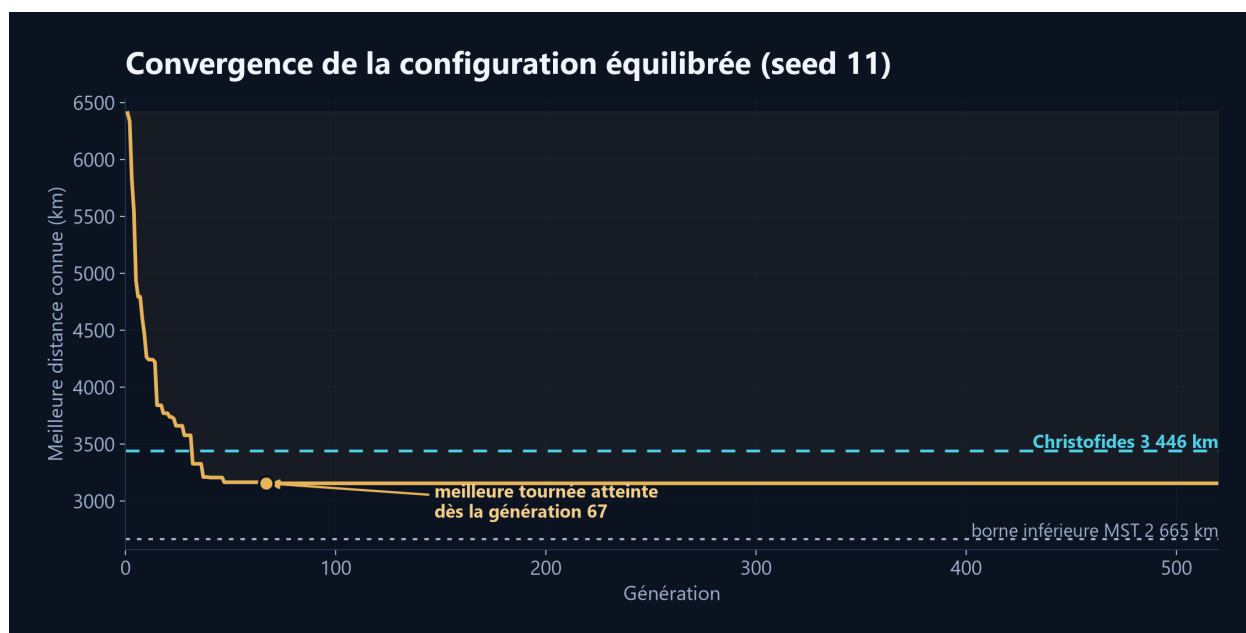
→ parent

k trop grand → pression trop forte, diversité perdue.
k trop petit → la sélection devient presque aléatoire.

Mutation par échange, mutation par inversion, et sélection par tournoi.



Le benchmark : trois configurations, trois seeds. Seule l'équilibrée est stable.



La convergence de la configuration retenue — meilleure tournée dès la génération 67.

Itinéraire algorithme génétique

Configuration équilibrée · meilleure tournée observée



L'itinéraire final de l'algorithme génétique — 3 157,13 km.

A.4 Comparaison et conclusion

Deux philosophies opposées

Le sujet impose de les implémenter toutes les deux — et de les comparer.

VS

CHRISTOFIDES

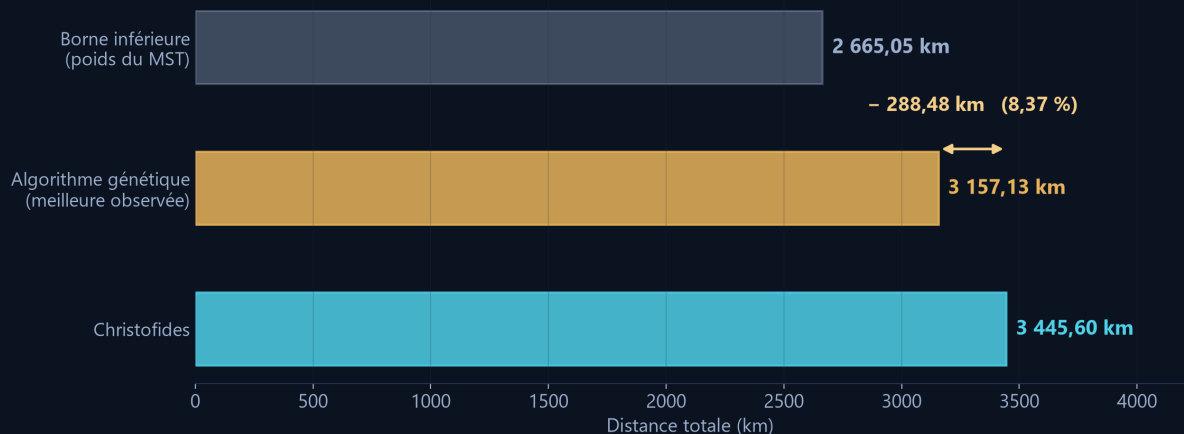
- Théorie des graphes
- Déterministe : même entrée, même sortie
- Une seule exécution suffit
- Garantie prouvée $\leq 1,5 \times \text{OPT}$
- Presque aucun paramètre à régler

ALGORITHME GÉNÉTIQUE

- Métaheuristique inspirée de l'évolution
- Stochastique : dépend du hasard (seed)
- Plusieurs exécutions nécessaires
- Aucune garantie d'optimalité
- 6 hyperparamètres à calibrer

Les deux philosophies mises en regard.

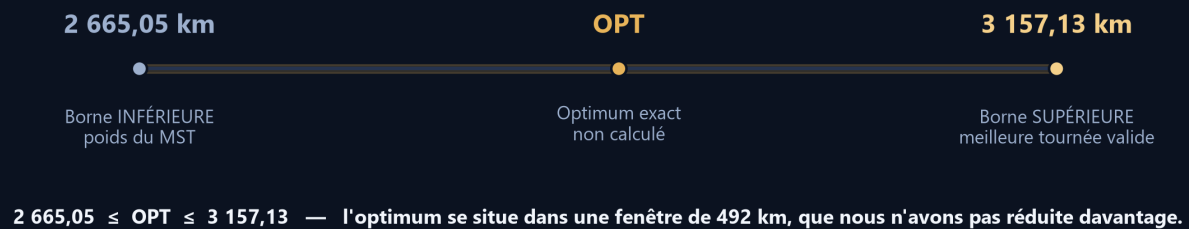
Comparaison des distances obtenues



Comparaison des distances, avec la borne inférieure du MST.

Ce que nous savons réellement de l'optimum

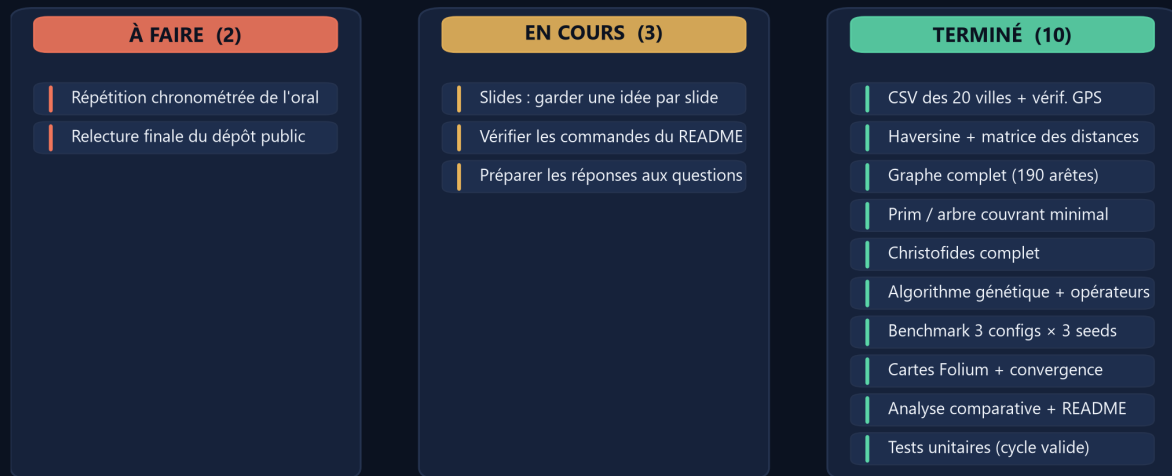
Nous ne l'avons pas calculé — nous l'encadrons.



Ce que nous savons réellement de l'optimum : un encadrement, pas une valeur.

Organisation du projet — tableau Kanban

Représentation de notre tableau Trello d'organisation.



Romain · Yannis · Lisa — revue du tableau à chaque séance, une carte = un livrable vérifiable.

L'organisation du projet, telle que suivie sur notre tableau Trello.

Document généré à partir de l'exécution réelle du projet. Toutes les valeurs sont vérifiables dans `results/summary.json` et reproductibles par `python main.py`.